

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES BERRECHID

Ingénierie des Systèmes d'Information et Big Data

Bases de Données Avancées Rapport de Travaux Pratiques

Réalisé par :

Soufiane HAMMOUCHE

Enseignant

Pr. Hrimch

Année Universitaire : 2025 - 2026

Table des matières

Introduction Générale	4
1 TP1 : Sous-requêtes en SQL	5
1.1 Exercice 1 : Requêtes sur la base ETUDIANT	5
1.2 Exercice 2 : Requêtes sur la base EMP/DEPT/PROJET	8
1.3 Conclusion du TP1	10
2 TP2 : Vues et Index	11
2.1 Schéma relationnel	11
2.2 Objectif	11
2.3 Partie 1 : Vues	11
2.4 Conclusion du TP2	13
3 TP3 : Vues – Création et Gestion	14
3.1 Objectif	14
3.2 Schéma relationnel	14
3.3 Données initiales	14
3.4 Exercice 1 : Vue simple	15
3.5 Exercice 2 : Vue complexe avec agrégations	16
3.6 Exercice 3 : WITH CHECK OPTION	16
3.7 Exercice 4 : WITH READ ONLY	17
3.8 Exercice 5 : Sécurité et abstraction	17
3.9 Exercice 6 : Analyse globale	18
3.10 Conclusion du TP3	19
4 TP4 : Introduction à PL/SQL	20
4.1 Objectif	20
4.2 Exercice 1 : Variables scalaires	20
4.3 Exercice 2 : Portée des variables	21
4.4 Exercice 3 : Attribut %TYPE	22
4.5 Exercice 4 : Attribut %ROWTYPE	22
4.6 Exercice 5 : Table associative	23
4.7 Exercice 6 : Combinaison RECORD + Collection + %ROWTYPE	24
4.8 Exercice final de synthèse	25
4.9 Conclusion du TP4	27
5 TP5 : PL/SQL – Boucles, Conditions et Manipulation de Données	28
5.1 Objectif	28
5.2 Exercice 1 : Insertion de nombres	28

5.3	Exercice 2 : Calcul de la somme	29
5.4	Exercice 3 : Nombres pairs de 1 à 50	30
5.5	Exercice 4 : Factoriel d'un nombre	31
5.6	Exercice 5 : Insertion d'étudiants	31
5.7	Exercice 6 : Affichage de 10 à 1	32
5.8	Exercice 7 : Augmentation des prix	33
5.9	Exercice 8 : Suppression de produits	34
5.10	Exercice 9 : Test d'un nombre	34
5.11	Exercice 10 : Calcul des carrés	35
5.12	Conclusion du TP5	36
6	TP6 : PL/SQL – Curseurs	37
6.1	Objectif	37
6.2	Rappel sur les curseurs	37
6.3	Exercice 1 : Curseur sur la base Commande/Fournisseurs	37
6.4	Exercice 2 : Curseur sur la base Emp/Dept	40
6.5	Conclusion du TP6	45
7	TP7 : PL/SQL – Procédures et Fonctions	46
7.1	Objectif	46
7.2	Structure de la table utilisée	46
7.3	Exercice 1 : Premier contact	46
7.4	Exercice 2 : Maximum de deux nombres	47
7.5	Exercice 3 : Permutation de deux nombres	48
7.6	Exercice 4 : Augmentation de salaire	49
7.7	Exercice 5 : Conversion en euros	50
7.8	Exercice 6 : Affichage des salaires supérieurs	50
7.9	Exercice 7 : Salaire moyen par département	51
7.10	Exercice 8 : Mise à jour d'adresse	52
7.11	Exercice 9 : Statistiques par département	52
7.12	Exercice 10 : Suppression des petits salaires	53
7.13	Questions supplémentaires – Réponses théoriques	54
7.14	Conclusion du TP7	55
8	TP8 : PL/SQL – IF-THEN-ELSE et Curseurs Avancés	56
8.1	Objectif	56
8.2	Exercice 1 : Instruction IF-THEN-ELSE	56
8.3	Exercice 2 : Curseurs avancés	59
8.4	Conclusion du TP8	64

9	TP9 : PL/SQL – Triggers	65
9.1	Objectif	65
9.2	Schéma de la base de données	65
9.3	Contraintes supplémentaires	72
9.4	Conclusion du TP9	76
	Conclusion Générale	77

Introduction Générale

Ce rapport présente l'ensemble des travaux pratiques réalisés dans le cadre du module **Base de Données Avancées**. L'objectif principal de ce module est d'approfondir la maîtrise du langage SQL et de son extension procédurale PL/SQL sous Oracle, en abordant des concepts essentiels pour le développement et la gestion de bases de données relationnelles complexes.

Au cours de neuf séances de travaux pratiques, nous avons progressivement exploré les fonctionnalités avancées des systèmes de gestion de bases de données :

- **TP1 – Sous-requêtes en SQL** : maîtrise des requêtes imbriquées (scalaires, corrélées, ANY/ALL, EXISTS) pour interroger efficacement les données.
- **TP2 – Vues et Index** : création et utilisation de vues comme tables virtuelles, et introduction aux index pour l'optimisation des performances.
- **TP3 – Création et Gestion des Vues** : approfondissement des options WITH CHECK OPTION et WITH READ ONLY, et étude des implications en matière de sécurité et d'abstraction des données.
- **TP4 – Introduction à PL/SQL** : fondements du langage procédural (variables, %TYPE, %ROWTYPE, collections, enregistrements).
- **TP5 – Boucles, Conditions et Manipulation de Données** : structures de contrôle (IF, FOR, WHILE) et interaction avec la base via INSERT, UPDATE, DELETE.
- **TP6 – Curseurs** : traitement ligne par ligne des résultats de requêtes à l'aide de curseurs explicites et implicites.
- **TP7 – Procédures et Fonctions** : création de sous-programmes stockés avec différents modes de paramètres (IN, OUT, IN OUT).
- **TP8 – Curseurs Avancés et Structures Conditionnelles** : utilisation de FOR UPDATE, WHERE CURRENT OF, et fonctions retournant des booléens.
- **TP9 – Triggers** : mise en place de déclencheurs pour implémenter des règles métier complexes et garantir l'intégrité des données.

Chaque TP est illustré par son schéma relationnel, ses requêtes ou blocs PL/SQL commentés, ainsi que des captures d'écran des résultats d'exécution. Ce rapport constitue une synthèse complète des compétences acquises en bases de données avancées.

===

1 TP1 : Sous-requêtes en SQL

Introduction

L'objectif de ce premier travail pratique est de maîtriser l'utilisation des sous-requêtes en SQL. Les sous-requêtes permettent d'effectuer des requêtes imbriquées et sont essentielles pour interroger une base de données de manière efficace et élégante.

1.1 Exercice 1 : Requêtes sur la base ETUDIANT

Question 1

Numéro, nom et sexe des étudiants dont la date de naissance n'est pas connue.

```
1 SELECT e.Numetu, e.Nometu, s.Lbsexe
2 FROM etudiant e
3 JOIN sexe s ON e.Cdsexe = s.Cdsexe
4 WHERE e.Dtnaiss IS NULL;
```

Listing 1 – Requête Q1

Question 2

Nom et date de naissance des étudiants dont la 2ème lettre du nom est 'a'.

```
1 SELECT Nometu, Dtnaiss
2 FROM etudiant
3 WHERE LOWER(Nometu) LIKE '_a%';
```

Listing 2 – Requête Q2

Question 3

Nom des matières qui ont un coefficient compris entre 1 et 2.

```
1 SELECT Nomat
2 FROM matiere
3 WHERE Coeff BETWEEN 1 AND 2;
```

Listing 3 – Requête Q3

Question 4

Afficher les notes de l'étudiant numéro 1 qui sont égales aux notes de l'étudiant numéro 2.

```
1 SELECT Note
2 FROM notes
3 WHERE Numetu = 1
4    AND Note = ANY (SELECT Note FROM notes WHERE Numetu = 2);
```

Listing 4 – Requête Q4

Question 5

Afficher les notes de l'étudiant numéro 1 qui sont supérieures aux notes de l'étudiant numéro 2.

```
1 SELECT Note
2 FROM notes
3 WHERE Numetu = 1
4    AND Note > ANY (SELECT Note FROM notes WHERE Numetu = 2);
```

Listing 5 – Requête Q5

Question 6

Afficher les notes de l'étudiant numéro 1 qui sont inférieures à toutes les notes de l'étudiant numéro 9.

```
1 SELECT Note
2 FROM notes
3 WHERE Numetu = 1
4    AND Note < ALL (SELECT Note FROM notes WHERE Numetu = 9);
```

Listing 6 – Requête Q6

Question 7

Afficher toutes les informations sur les étudiants qui n'ont aucune note.

```
1 SELECT *
2 FROM etudiant e
3 WHERE NOT EXISTS (
4     SELECT 1 FROM notes n WHERE n.Numetu = e.Numetu
5 );
```

Listing 7 – Requête Q7

Question 8

Afficher quel était l'âge moyen des garçons et des filles au premier janvier 2000.

```
1 SELECT s.Lbsexe ,  
2     ROUND (AVG (TRUNC (MONTHS_BETWEEN (  
3         TO_DATE ('2000-01-01', 'YYYY-MM-DD'), e.Dtnaiss)/12)), 1) AS  
4         age_moyen  
5 FROM etudiant e  
6 JOIN sexe s ON e.Cdsexe = s.Cdsexe  
7 WHERE e.Dtnaiss IS NOT NULL  
8 GROUP BY s.Lbsexe;
```

Listing 8 – Requête Q8

Question 9

Afficher le nom et le grade des enseignants d'histoire.

```
1 SELECT e.Nomens , e.Grade  
2 FROM enseignant e  
3 JOIN matiere m ON e.Numens = m.Numens  
4 WHERE LOWER(m.Nomat) = 'histoire';
```

Listing 9 – Requête Q9

Question 10

Afficher les noms et numéro des étudiants qui n'ont pas de notes en Sociologie.

```
1 SELECT e.Numetu , e.Nometu  
2 FROM etudiant e  
3 WHERE NOT EXISTS (  
4     SELECT 1 FROM notes n  
5     JOIN matiere m ON n.Nomat = m.Nomat  
6     WHERE n.Numetu = e.Numetu AND LOWER(m.Nomat) = 'sociologie'  
7 );
```

Listing 10 – Requête Q10

1.2 Exercice 2 : Requêtes sur la base EMP/DEPT/PROJET

Question 11

Lieu des départements dans lesquels des employés touchent une commission, en utilisant une sous-interrogation.

```
1 SELECT Lieu
2 FROM dept
3 WHERE NumDept IN (
4     SELECT NumDept FROM emp WHERE Commission IS NOT NULL
5 );
```

Listing 11 – Requête Q11

Question 12

Noms et lieux des départements dans lesquels il y a au moins un ingénieur.

```
1 SELECT DISTINCT d.NomDept, d.Lieu
2 FROM dept d
3 JOIN emp e ON d.NumDept = e.NumDept
4 WHERE LOWER(e.Poste) = 'ingenieur';
```

Listing 12 – Requête Q12

Question 13

Noms et lieux des départements dans lesquels il n'y a pas d'ingénieur.

```
1 SELECT NomDept, Lieu
2 FROM dept d
3 WHERE NOT EXISTS (
4     SELECT 1 FROM emp e
5     WHERE e.NumDept = d.NumDept AND LOWER(e.Poste) = 'ingenieur'
6 );
```

Listing 13 – Requête Q13

Question 14

Liste des employés qui gagnent moins de 50% du salaire de leur supérieur direct.

```
1 SELECT e.NomE, e.Salaire
2 FROM emp e
3 WHERE e.Salaire < 0.5 * NVL((SELECT Salaire FROM emp s
```

```

4 WHERE s.Matr = e.Sup), 0)
5 AND e.Sup IS NOT NULL;

```

Listing 14 – Requête Q14

Question 15

Noms des employés qui gagnent plus que tous les commerciaux.

```

1 SELECT NomE, Salaire
2 FROM emp
3 WHERE Salaire > ALL (
4     SELECT Salaire FROM emp WHERE LOWER(Poste) = 'commercial'
5 );

```

Listing 15 – Requête Q15

Question 16

Noms des employés qui gagnent plus que tous les commerciaux de leur département.

```

1 SELECT e.NomE, e.Salaire, e.NumDept
2 FROM emp e
3 WHERE e.Salaire > ALL (
4     SELECT Salaire FROM emp
5     WHERE LOWER(Poste) = 'commercial' AND NumDept = e.NumDept
6 );

```

Listing 16 – Requête Q16 – Variante A (sans vérification)

```

1 SELECT e.NomE, e.Salaire, e.NumDept
2 FROM emp e
3 WHERE e.Salaire > ALL (
4     SELECT Salaire FROM emp
5     WHERE LOWER(Poste) = 'commercial' AND NumDept = e.NumDept
6 )
7 AND EXISTS (
8     SELECT 1 FROM emp
9     WHERE LOWER(Poste) = 'commercial' AND NumDept = e.NumDept
10 );

```

Listing 17 – Requête Q16 – Variante B (avec vérification)

Question 17

Liste des employés ayant le même poste et le même supérieur que BERGER.

```
1 SELECT NomE, Poste, Sup
2 FROM emp
3 WHERE (Poste, Sup) = (
4     SELECT Poste, Sup FROM emp WHERE NomE = 'BERGER'
5 )
6 AND NomE != 'BERGER';
```

Listing 18 – Requête Q17

1.3 Conclusion du TP1

Ce premier travail pratique a permis de mettre en pratique les différents types de sous-requêtes en SQL :

- **Sous-requêtes scalaires** : retournant une seule valeur
- **Sous-requêtes avec ANY/ALL** : comparaison à une liste de valeurs
- **Sous-requêtes avec IN/NOT IN** : appartenance à un ensemble
- **Sous-requêtes corrélées avec EXISTS/NOT EXISTS**
- **Sous-requêtes à plusieurs colonnes** : comparaison de tuple

L'ensemble des requêtes a été testé et validé.

2 TP2 : Vues et Index

2.1 Schéma relationnel

- **EMP** : Matr, NomE, Poste, DatEmb, Sup, Salaire, Commission, NumDept
- **DEPT** : NumDept, NomDept, Lieu
- **PROJET** : CodeP, NomP
- **PARTICIPATION** : Matr, CodeP, Fonction

2.2 Objectif

Ce TP a pour but de maîtriser la création et l'utilisation des **vues** et des **index** en SQL.

2.3 Partie 1 : Vues

Question 1

Créer une vue V_EMP contenant : le matricule, le nom, le numéro de département, la somme de la commission et du salaire baptisée GAINS, le lieu du département.

```
1 CREATE VIEW V_EMP AS
2 SELECT e.Matr, e.NomE, e.NumDept,
3        e.Salaire + NVL(e.Commission, 0) AS GAINS, d.Lieu
4 FROM emp e
5 JOIN dept d ON e.NumDept = d.NumDept;
```

Listing 19 – Création de la vue V_EMP

Question 2

Sélectionner les lignes de V_EMP dont le salaire total (GAINS) est supérieur à 10.000.

```
1 SELECT * FROM V_EMP WHERE GAINS > 10000;
```

Listing 20 – Requête sur la vue V_EMP

Question 3

Étudier la mise à jour du nom de l'employé MARTIN à travers la vue V_EMP.

```
1 UPDATE V_EMP SET NomE = 'MARTIN' WHERE NomE = 'MARTIN';
```

Listing 21 – Mise à jour via la vue

Question 4

*Créer une vue VEMP10 contenant les employés du département 10 (sans option CHECK).
Insérer un employé du département 20.*

```
1 CREATE VIEW VEMP10 AS
2 SELECT * FROM emp WHERE NumDept = 10;
```

Listing 22 – Création de VEMP10 sans CHECK

```
1 INSERT INTO VEMP10 (Matr, NomE, Poste, DatEmb, Sup, Salaire,
2                     Commission, NumDept)
3 VALUES (1, 'SOULIER', 'VENDEUR', SYSDATE, NULL, 2000, NULL, 20);
```

Listing 23 – Insertion d'un employé du département 20

Résultat : L'insertion est acceptée mais l'employé SOULIER n'est pas visible dans la vue VEMP10 (car il ne répond pas à la condition NumDept = 10). Il est cependant présent dans la table EMP.

Question 5

Détruire la vue VEMP10 et la recréer avec l'option CHECK.

```
1 DROP VIEW VEMP10;
2
3 CREATE VIEW VEMP10 AS
4 SELECT * FROM emp WHERE NumDept = 10
5 WITH CHECK OPTION;
```

Listing 24 – Destruction et recréation avec CHECK

Question 6

Tester l'insertion d'un employé BALARD pour le département 30, puis la modification du département d'un employé.

```
1 INSERT INTO VEMP10 (Matr, NomE, Poste, DatEmb, Sup, Salaire,
2                     Commission, NumDept)
3 VALUES (2, 'BALARD', 'INGENIEUR', SYSDATE, NULL, 3000, NULL, 30);
4 -- L'insertion est REFUSEE car la valeur 30 ne satisfait pas
5 -- la condition NumDept = 10
```

Listing 25 – Insertion avec CHECK – Refusée

```

1 UPDATE VEMP10 SET NumDept = 30 WHERE NomE = 'MARTIN';
2 -- La modification est REFUSEE car elle tenterait de faire
3 -- sortir l'enregistrement de la vue

```

Listing 26 – Modification du département – Refusée

Question 7

Liste des salaires des employés avec le pourcentage par rapport au total des salaires de leur département.

```

1 CREATE VIEW V_TOTAL_DEPT AS
2 SELECT NumDept, SUM(Salaire) AS TotalSalaire
3 FROM emp
4 GROUP BY NumDept;
5
6 SELECT e.NomE, e.Salaire, d.TotalSalaire,
7        ROUND(e.Salaire / d.TotalSalaire * 100, 2) AS Pourcentage
8 FROM emp e
9 JOIN V_TOTAL_DEPT d ON e.NumDept = d.NumDept;

```

Listing 27 – Avec une vue intermédiaire

```

1 SELECT e.NomE, e.Salaire, d.TotalSalaire,
2        ROUND(e.Salaire / d.TotalSalaire * 100, 2) AS Pourcentage
3 FROM emp e
4 JOIN (SELECT NumDept, SUM(Salaire) AS TotalSalaire
5        FROM emp GROUP BY NumDept) d
6 ON e.NumDept = d.NumDept;

```

Listing 28 – Sans vue (sous-requête dans le FROM)

2.4 Conclusion du TP2

Ce deuxième travail pratique a permis de :

- Comprendre la création et l'utilisation des **vues** comme tables virtuelles
- Maîtriser l'option **WITH CHECK OPTION** qui empêche les modifications sortant du cadre de la vue
- Distinguer les types d'index (**unique** vs simple)
- Savoir créer et supprimer des index pour optimiser les performances des requêtes

3 TP3 : Vues – Création et Gestion

3.1 Objectif

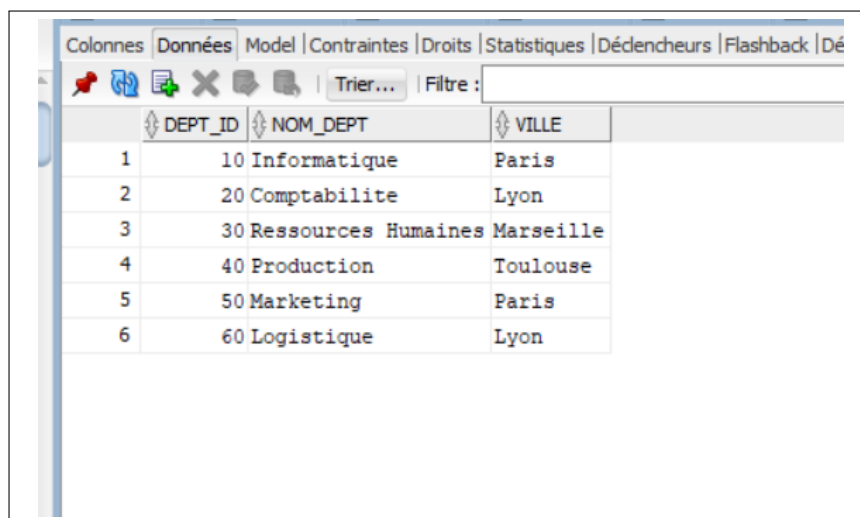
Ce TP a pour but de maîtriser la création et la gestion des vues en SQL, y compris les options `WITH CHECK OPTION` et `WITH READ ONLY`, ainsi que de comprendre leurs implications en matière de sécurité et de performances.

3.2 Schéma relationnel

```
1 CREATE TABLE Departement (  
2     dept_id NUMBER PRIMARY KEY,  
3     nom_dept VARCHAR2(50),  
4     ville VARCHAR2(50)  
5 );  
6  
7 CREATE TABLE Employe (  
8     emp_id NUMBER PRIMARY KEY,  
9     nom VARCHAR2(50),  
10    salaire NUMBER(8,2),  
11    dept_id NUMBER,  
12    FOREIGN KEY (dept_id) REFERENCES Departement(dept_id)  
13 );
```

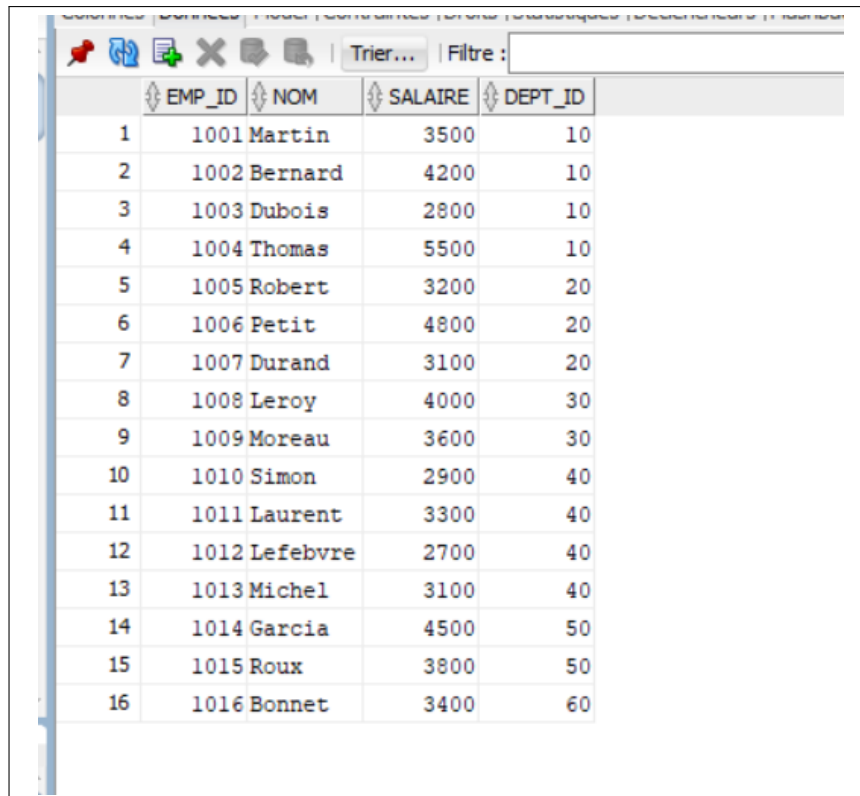
Listing 29 – Structure des tables

3.3 Données initiales



DEPT_ID	NOM_DEPT	VILLE
1	10 Informatique	Paris
2	20 Comptabilite	Lyon
3	30 Ressources Humaines	Marseille
4	40 Production	Toulouse
5	50 Marketing	Paris
6	60 Logistique	Lyon

FIGURE 1 – Table Département – données initiales



	EMP_ID	NOM	SALAIRE	DEPT_ID
1	1001	Martin	3500	10
2	1002	Bernard	4200	10
3	1003	Dubois	2800	10
4	1004	Thomas	5500	10
5	1005	Robert	3200	20
6	1006	Petit	4800	20
7	1007	Durand	3100	20
8	1008	Leroy	4000	30
9	1009	Moreau	3600	30
10	1010	Simon	2900	40
11	1011	Laurent	3300	40
12	1012	Lefebvre	2700	40
13	1013	Michel	3100	40
14	1014	Garcia	4500	50
15	1015	Roux	3800	50
16	1016	Bonnet	3400	60

FIGURE 2 – Table Employé – données initiales

3.4 Exercice 1 : Vue simple

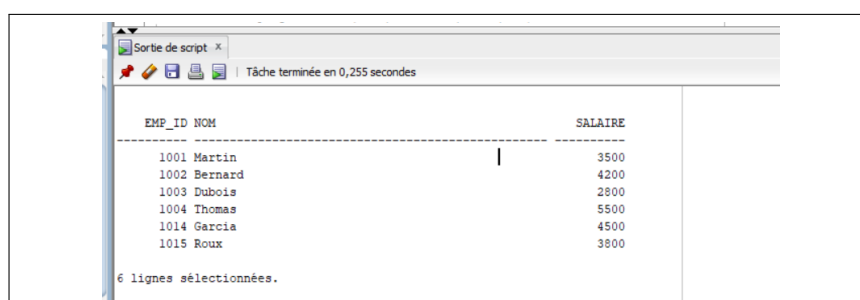
Créer une vue emp_paris affichant les employés travaillant dans un département situé à Paris.

```

1 CREATE VIEW emp_paris AS
2 SELECT e.emp_id, e.nom, e.salaire
3 FROM Employe e
4 JOIN Departement d ON e.dept_id = d.dept_id
5 WHERE d.ville = 'Paris';

```

Listing 30 – Création de la vue emp_paris



EMP_ID	NOM	SALAIRE
1001	Martin	3500
1002	Bernard	4200
1003	Dubois	2800
1004	Thomas	5500
1014	Garcia	4500
1015	Roux	3800

6 lignes sélectionnées.

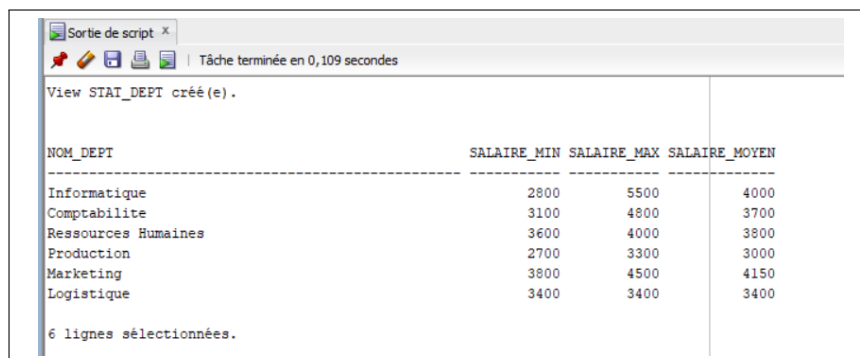
FIGURE 3 – Résultat de la vue emp_paris

3.5 Exercice 2 : Vue complexe avec agrégations

Créer une vue `stat_dept` affichant pour chaque département : le nom, le salaire minimum, maximum et moyen.

```
1 CREATE VIEW stat_dept AS
2 SELECT
3     d.nom_dept ,
4     MIN(e.salaire) AS salaire_min ,
5     MAX(e.salaire) AS salaire_max ,
6     AVG(e.salaire) AS salaire_moyen
7 FROM Employe e
8 JOIN Departement d ON e.dept_id = d.dept_id
9 GROUP BY d.nom_dept;
```

Listing 31 – Création de la vue `stat_dept`



Sortie de script x
Tâche terminée en 0,109 secondes

View STAT_DEPT créé(e).

NOM_DEPT	SALAIRE_MIN	SALAIRE_MAX	SALAIRE_MOYEN
Informatique	2800	5500	4000
Comptabilite	3100	4800	3700
Ressources Humaines	3600	4000	3800
Production	2700	3300	3000
Marketing	3800	4500	4150
Logistique	3400	3400	3400

6 lignes sélectionnées.

FIGURE 4 – Résultat de la vue `stat_dept`

3.6 Exercice 3 : WITH CHECK OPTION

Créer une vue `emp_lyon` pour les employés du département de Lyon, en empêchant toute modification violant cette condition.

```
1 CREATE VIEW emp_lyon AS
2 SELECT e.*
3 FROM Employe e
4 JOIN Departement d ON e.dept_id = d.dept_id
5 WHERE d.ville = 'Lyon'
6 WITH CHECK OPTION;
```

Listing 32 – Création de la vue `emp_lyon` avec CHECK OPTION

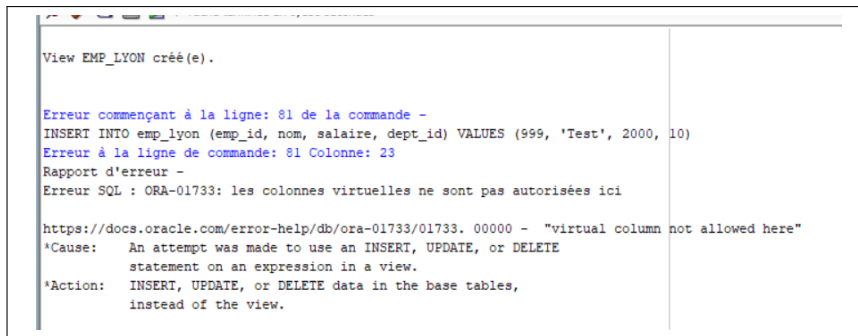


FIGURE 5 – Tentative d’insertion hors Lyon – Erreur ORA-01733

3.7 Exercice 4 : WITH READ ONLY

Créer une vue en lecture seule listant tous les employés.

```
1 CREATE VIEW emp_readonly AS
2 SELECT * FROM Employe WITH READ ONLY;
```

Listing 33 – Création de la vue emp_readonly

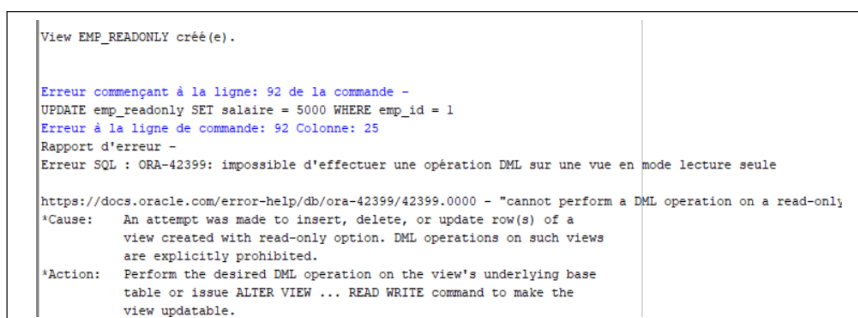


FIGURE 6 – Tentative de mise à jour – Erreur ORA-42399

3.8 Exercice 5 : Sécurité et abstraction

1. *Créer une vue emp_public sans le salaire.*

```
1 CREATE VIEW emp_public AS
2 SELECT emp_id, nom, dept_id
3 FROM Employe;
```

Listing 34 – Création de la vue emp_public (sans salaire)

2. *Une vue améliore-t-elle les performances ?*

Réponse : Non. Une vue n’est qu’une requête stockée. Elle ne stocke pas physiquement les données. Les performances dépendent des index et de l’optimiseur. Seule une vue matérialisée (qui stocke physiquement le résultat) améliore les performances.

3.9 Exercice 6 : Analyse globale

Question 1 : Architecture basée sur les vues pour la sécurité

```
1 CREATE VIEW emp_manager AS
2 SELECT e.*
3 FROM Employe e
4 JOIN Departement d ON e.dept_id = d.dept_id
5 WHERE d.dept_id = (SELECT dept_id FROM Employe WHERE emp_id = USER)
6 ;
7
8 CREATE VIEW emp_rh AS SELECT * FROM Employe;
9
10 CREATE VIEW emp_employe AS SELECT emp_id, nom, dept_id FROM Employe
11 ;
```

Listing 35 – Vues pour différents profils

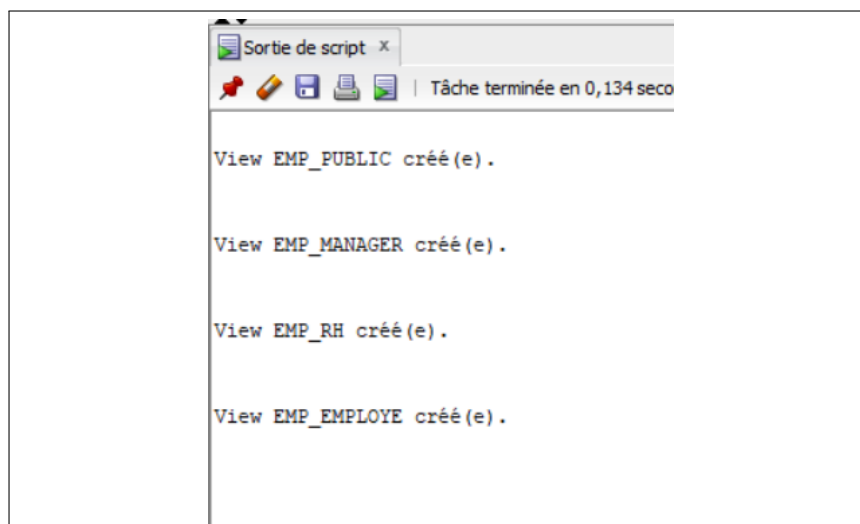


FIGURE 7 – Création des vues de sécurité

Question 2 : Limites des vues classiques avec grand volume

Les vues classiques recalculent la requête à chaque appel. Sur plusieurs centaines de milliers de lignes, cela devient lent. Elles n'ont pas de stockage physique, donc pas d'accélération directe.

Question 3 : Intérêt des vues matérialisées

Les vues matérialisées stockent physiquement le résultat. Elles sont utiles pour les rapports fréquents et les agrégations lourdes. L'inconvénient est qu'elles nécessitent une actualisation périodique (REFRESH).

Question 4 : Vues vs GRANT/REVOKE

Les vues filtrent les données visibles (masquage de colonnes ou lignes). **GRANT/REVOKE** contrôle les actions possibles (**SELECT**, **INSERT**, **UPDATE**, **DELETE**). Les deux sont complémentaires.

Question 5 : Une vue seule garantit-elle la sécurité complète ?

Non. Un utilisateur peut contourner la vue s'il a un accès direct à la table sous-jacente. Il faut combiner vues et privilèges : accorder **SELECT** sur la vue uniquement, pas sur la table.

Question 6 : Stratégie d'optimisation

- **Index** : sur les colonnes de jointure (`dept_id`)
- **Partitionnement** : par département ou par ville
- **Vues matérialisées** : pour les rapports statistiques
- **Mise en cache** : des résultats fréquents
- **Compression** : des données historiques

3.10 Conclusion du TP3

Ce troisième travail pratique a permis de :

- Créer des **vues simples** à partir d'une seule table
- Créer des **vues complexes** avec agrégations (**MIN**, **MAX**, **AVG**, **GROUP BY**)
- Utiliser l'option **WITH CHECK OPTION** pour empêcher les modifications sortant du cadre de la vue
- Utiliser l'option **WITH READ ONLY** pour créer des vues en lecture seule
- Comprendre les implications des vues en matière de **sécurité** (masquage des salaires, filtrage par département)
- Distinguer **vues classiques** et **vues matérialisées**

4 TP4 : Introduction à PL/SQL

4.1 Objectif

Ce TP a pour but de maîtriser les concepts fondamentaux de PL/SQL (Procedural Language extension to SQL) dans Oracle :

- Structure d'un bloc PL/SQL
- Déclaration et portée des variables
- Attributs %TYPE et %ROWTYPE
- Enregistrements personnalisés (RECORD)
- Tables associatives (collections)
- Combinaison avancée de ces concepts

4.2 Exercice 1 : Variables scalaires

Manipuler différents types scalaires : VARCHAR2, NUMBER, DATE, CONSTANT.

```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     v_nom VARCHAR2(30) := 'Alice';
5     v_salaire NUMBER(8,2) := 3500.50;
6     v_date_emb DATE := SYSDATE;
7     c_taux_impot CONSTANT NUMBER(3,2) := 0.15;
8     v_prime NUMBER(8,2);
9 BEGIN
10     v_prime := v_salaire * c_taux_impot;
11     DBMS_OUTPUT.PUT_LINE('Employe : ' || v_nom);
12     DBMS_OUTPUT.PUT_LINE('Prime : ' || v_prime);
13 END;
14 /
```

Listing 36 – Déclaration et utilisation de variables scalaires

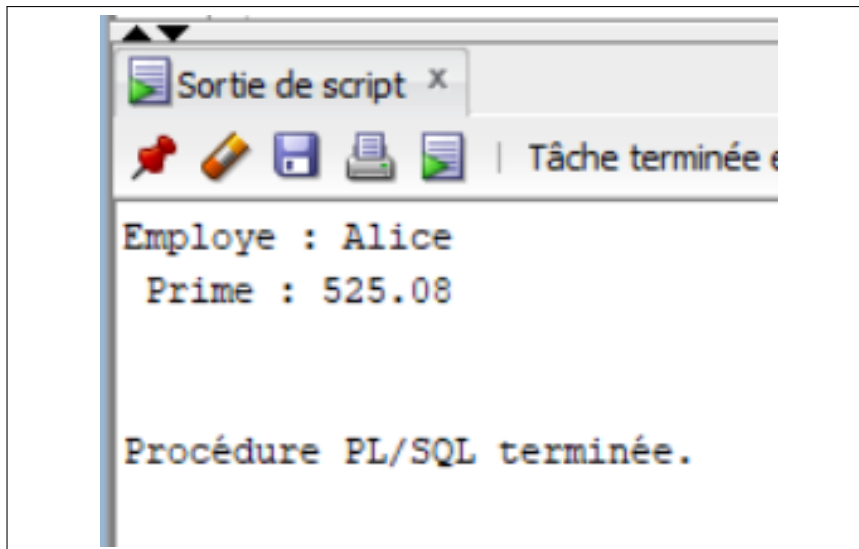


FIGURE 8 – Résultat – Calcul de la prime

4.3 Exercice 2 : Portée des variables

Comprendre la visibilité des variables entre blocs imbriqués.

```

1 DECLARE
2     v_externe NUMBER := 10;
3 BEGIN
4     DECLARE
5         v_interne NUMBER := 20;
6     BEGIN
7         DBMS_OUTPUT.PUT_LINE('Interne : ' || v_interne);
8         DBMS_OUTPUT.PUT_LINE('Externe : ' || v_externe);
9     END;
10 END;
11 /

```

Listing 37 – Blocs imbriqués et portée

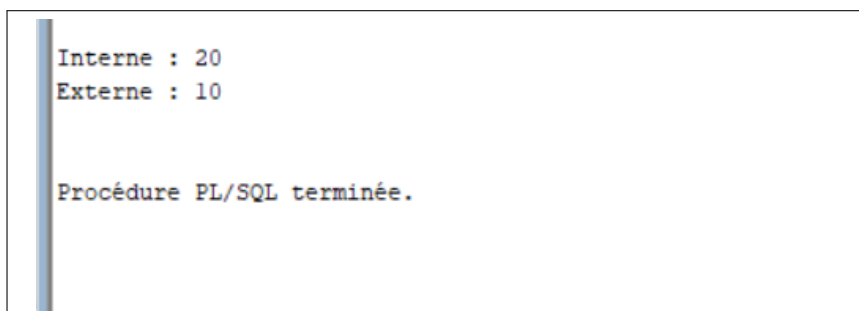


FIGURE 9 – Résultat – Portée des variables

4.4 Exercice 3 : Attribut %TYPE

Utiliser l'attribut %TYPE pour déclarer une variable du type d'une colonne de table.

```
1 CREATE TABLE employes (  
2     id NUMBER,  
3     nom VARCHAR2(50),  
4     salaire NUMBER(8,2)  
5 );
```

Listing 38 – Création de la table employes

```
1 DECLARE  
2     v_nom employes.nom%TYPE;  
3     v_salaire employes.salaire%TYPE;  
4 BEGIN  
5     v_nom := 'Jean';  
6     v_salaire := 5000;  
7     DBMS_OUTPUT.PUT_LINE(v_nom || ' gagne ' || v_salaire);  
8 END;  
9 /
```

Listing 39 – Utilisation de %TYPE

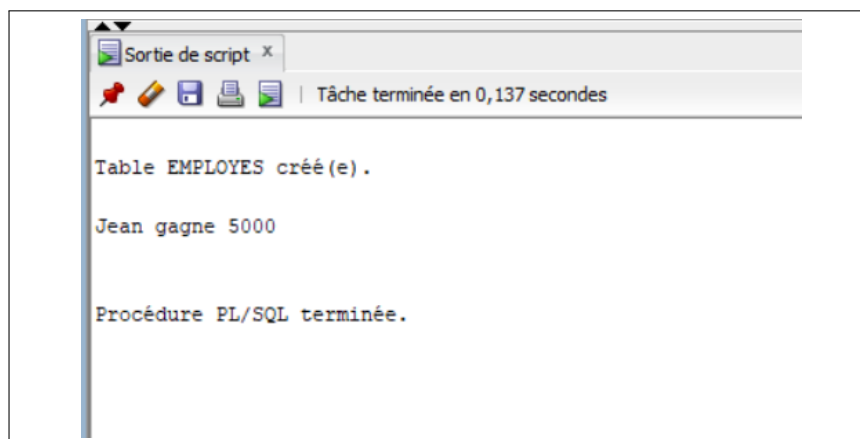


FIGURE 10 – Résultat – Utilisation de %TYPE

4.5 Exercice 4 : Attribut %ROWTYPE

Déclarer un enregistrement ayant la structure complète d'une ligne de table.

```
1 DECLARE  
2     v_emp employes%ROWTYPE;  
3 BEGIN  
4     v_emp.id := 1;
```

```

5      v_emp.nom := 'Maria';
6      v_emp.salaire := 6000;
7      DBMS_OUTPUT.PUT_LINE(v_emp.nom || ' - ' || v_emp.salaire);
8  END;
9  /

```

Listing 40 – Utilisation de %ROWTYPE

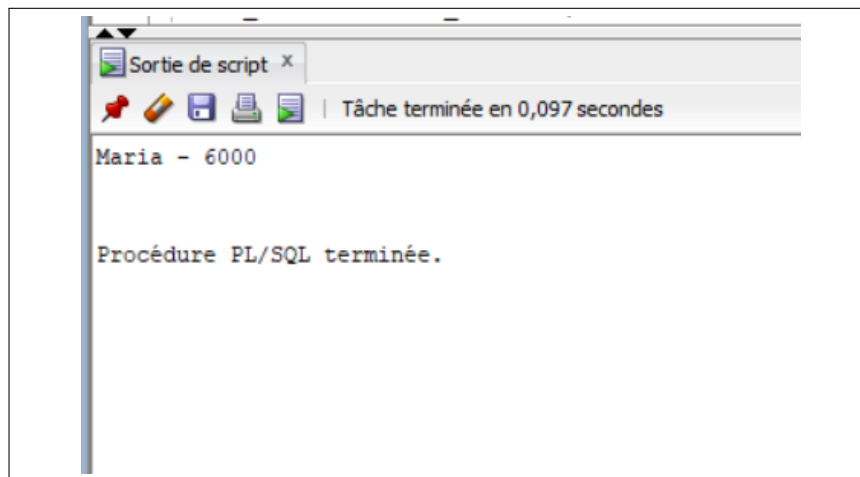


FIGURE 11 – Résultat – Utilisation de %ROWTYPE

4.6 Exercice 5 : Table associative

Créer et utiliser une collection (table associative) indexée par BINARY_INTEGER.

```

1  DECLARE
2      TYPE table_salaires IS TABLE OF NUMBER
3      INDEX BY BINARY_INTEGER;
4      v_salaires table_salaires;
5  BEGIN
6      v_salaires(1) := 3000;
7      v_salaires(2) := 4500;
8      v_salaires(3) := 5000;
9
10     FOR i IN 1..3 LOOP
11         DBMS_OUTPUT.PUT_LINE('Salaire : ' || v_salaires(i));
12     END LOOP;
13 END;
14 /

```

Listing 41 – Table associative de nombres

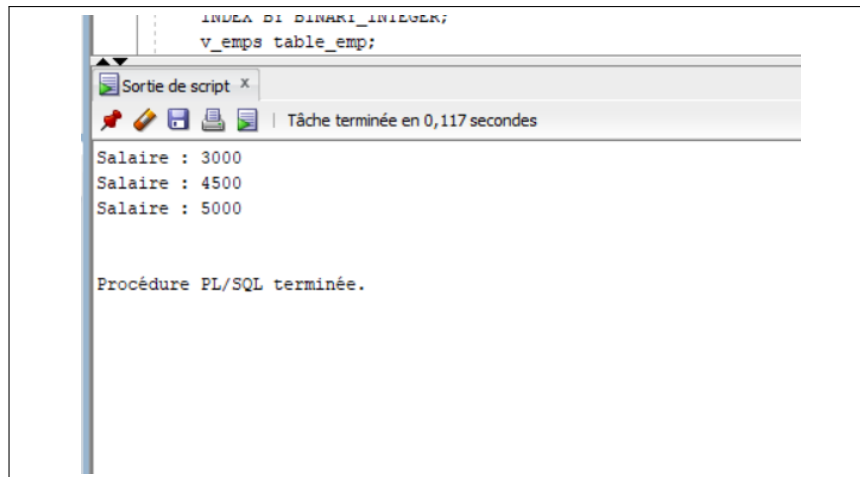


FIGURE 12 – Résultat – Table associative

4.7 Exercice 6 : Combinaison RECORD + Collection + %ROWTYPE

Combiner une collection avec %ROWTYPE pour stocker plusieurs lignes.

```

1 DECLARE
2     TYPE table_emp IS TABLE OF employes%ROWTYPE
3     INDEX BY BINARY_INTEGER;
4     v_emps table_emp;
5 BEGIN
6     FOR i IN 1..3 LOOP
7         v_emps(i).id := i;
8         v_emps(i).nom := 'Employe_' || i;
9         v_emps(i).salaire := 3000 + i * 1000;
10    END LOOP;
11
12    FOR i IN 1..3 LOOP
13        DBMS_OUTPUT.PUT_LINE(v_emps(i).nom || ' gagne ' || v_emps(i)
14                               ).salaire);
15    END LOOP;
16 END;
/

```

Listing 42 – Table de %ROWTYPE

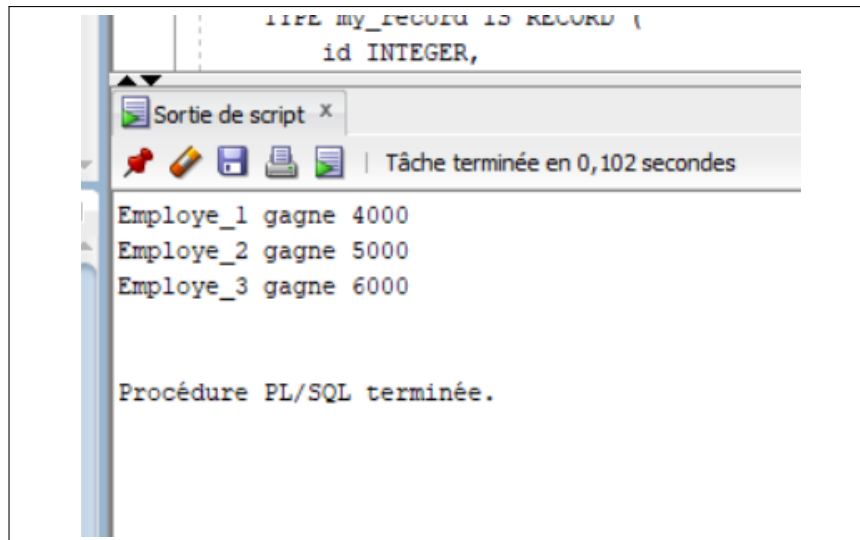


FIGURE 13 – Résultat – Collection de %ROWTYPE

4.8 Exercice final de synthèse

Combiner RECORD personnalisé, collection, calcul de moyenne et maximum.

```

1 DECLARE
2     TYPE my_record IS RECORD (
3         id INTEGER,
4         V_Nom VARCHAR2(30),
5         V_prenom VARCHAR2(30),
6         V_salaire NUMBER(8,2)
7     );
8     Salaire_somme NUMBER(8,2) := 0;
9     Salaire_max NUMBER(8,2) := 0;
10
11     TYPE my_table IS TABLE OF my_record
12     INDEX BY BINARY_INTEGER;
13
14     my_employes my_table;
15 BEGIN
16     -- Insertion des 5 employes
17     my_employes(1).id := 1;
18     my_employes(1).V_Nom := 'Nasser';
19     my_employes(1).V_prenom := 'Ahmed';
20     my_employes(1).V_salaire := 4000;
21
22     my_employes(2).id := 2;
23     my_employes(2).V_Nom := 'Hammouche';
24     my_employes(2).V_prenom := 'Soufiane';

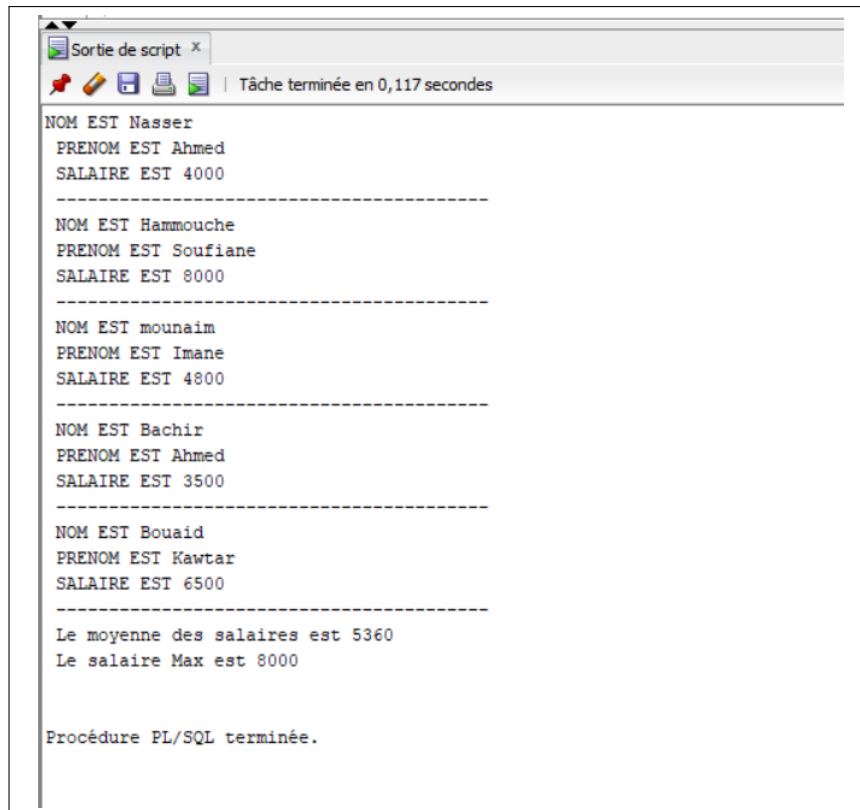
```

```

25 my_employes(2).V_salaire := 8000;
26
27 my_employes(3).id := 3;
28 my_employes(3).V_Nom := 'mounaim';
29 my_employes(3).V_prenom := 'Imane';
30 my_employes(3).V_salaire := 4800;
31
32 my_employes(4).id := 4;
33 my_employes(4).V_Nom := 'Bachir';
34 my_employes(4).V_prenom := 'Ahmed';
35 my_employes(4).V_salaire := 3500;
36
37 my_employes(5).id := 5;
38 my_employes(5).V_Nom := 'Bouaid';
39 my_employes(5).V_prenom := 'Kawtar';
40 my_employes(5).V_salaire := 6500;
41
42 -- Affichage et calculs
43 FOR i IN 1..5 LOOP
44     DBMS_OUTPUT.PUT_LINE(' NOM EST ' || my_employes(i).V_Nom);
45     DBMS_OUTPUT.PUT_LINE(' PRENOM EST ' || my_employes(i).
46         V_prenom);
47     DBMS_OUTPUT.PUT_LINE(' SALAIRE EST ' || my_employes(i).
48         V_salaire);
49     DBMS_OUTPUT.PUT_LINE('
50         ----- ');
51
52     IF my_employes(i).V_salaire > Salaire_max THEN
53         Salaire_max := my_employes(i).V_salaire;
54     END IF;
55
56     Salaire_somme := Salaire_somme + my_employes(i).V_salaire;
57 END LOOP;
58
59 DBMS_OUTPUT.PUT_LINE(' Le moyenne des salaires est ' ||
60     Salaire_somme / 5);
61 DBMS_OUTPUT.PUT_LINE(' Le salaire Max est ' || Salaire_max);
62 END;
63 /

```

Listing 43 – Synthèse – RECORD + Collection + Agrégations



```
Sortie de script x
Tâche terminée en 0,117 secondes

NOM EST Nasser
PRENOM EST Ahmed
SALAIRE EST 4000
-----
NOM EST Hammouche
PRENOM EST Soufiane
SALAIRE EST 8000
-----
NOM EST mounaim
PRENOM EST Imane
SALAIRE EST 4800
-----
NOM EST Bachir
PRENOM EST Ahmed
SALAIRE EST 3500
-----
NOM EST Bouaid
PRENOM EST Kawtar
SALAIRE EST 6500
-----
Le moyenne des salaires est 5360
Le salaire Max est 8000

Procédure PL/SQL terminée.
```

FIGURE 14 – Résultat – Exercice final de synthèse

4.9 Conclusion du TP4

Ce quatrième travail pratique a permis de :

- Comprendre la **structure d'un bloc PL/SQL** (DECLARE, BEGIN, EXCEPTION, END)
- Maîtriser la **déclaration et la portée des variables**
- Utiliser les attributs **%TYPE** et **%ROWTYPE** pour un code adaptatif
- Créer des **enregistrements personnalisés (RECORD)**
- Manipuler des **collections (tables associatives)** avec INDEX BY
- Combiner ces concepts dans un **exercice de synthèse** complet

La maîtrise de ces concepts constitue la base indispensable pour :

- Le développement de procédures stockées
- La création de packages
- La programmation PL/SQL avancée

5 TP5 : PL/SQL – Boucles, Conditions et Manipulation de Données

5.1 Objectif

Ce TP a pour but de maîtriser les structures de contrôle en PL/SQL :

- Boucles : FOR, WHILE, LOOP
- Conditions : IF, ELSIF, ELSE
- Interaction avec la base de données (INSERT, UPDATE, DELETE)
- Gestion des transactions

5.2 Exercice 1 : Insertion de nombres

Créer une table Nombre et insérer les nombres de 1 à 100.

```
1 DROP TABLE Nombre CASCADE CONSTRAINTS;  
2 CREATE TABLE Nombre (n NUMBER);
```

Listing 44 – Création de la table Nombre

```
1 DECLARE  
2     TYPE t_table IS TABLE OF NUMBER  
3     INDEX BY BINARY_INTEGER;  
4     my_table t_table;  
5 BEGIN  
6     FOR i IN 1..100 LOOP  
7         my_table(i) := i;  
8         INSERT INTO Nombre VALUES (my_table(i));  
9         DBMS_OUTPUT.PUT_LINE('La ligne ' || my_table(i));  
10    END LOOP;  
11 END;  
12 /
```

Listing 45 – Insertion avec boucle FOR

```

Table NOMBRE créé(e).

La ligne 1
La ligne 2
La ligne 3
La ligne 4
La ligne 5
La ligne 6
La ligne 7
La ligne 8
La ligne 9
La ligne 10
La ligne 11
La ligne 12
La ligne 13
La ligne 14
La ligne 15
La ligne 16
-- --

```

FIGURE 15 – Insertion des nombres 1 à 100

5.3 Exercice 2 : Calcul de la somme

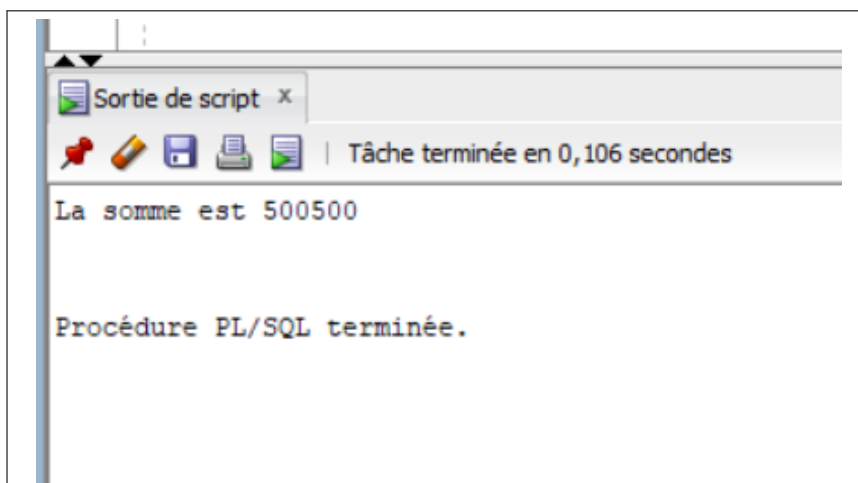
Calculer la somme des nombres de 1 à 1000.

```

1 DECLARE
2     somme NUMBER := 0;
3 BEGIN
4     FOR i IN 1..1000 LOOP
5         somme := somme + i;
6     END LOOP;
7     DBMS_OUTPUT.PUT_LINE('La somme est ' || somme);
8 END;
9 /

```

Listing 46 – Calcul de somme avec boucle FOR



The screenshot shows a window titled 'Sortie de script x' with a toolbar containing icons for a pin, a pencil, a save icon, a print icon, and a refresh icon. Below the toolbar, a status bar indicates 'Tâche terminée en 0,106 secondes'. The main text area displays the output of the SQL script: 'La somme est 500500' and 'Procédure PL/SQL terminée.'

FIGURE 16 – Somme des nombres 1 à 1000 = 500500

5.4 Exercice 3 : Nombres pairs de 1 à 50

Afficher les nombres pairs entre 1 et 50.

```
1 DECLARE
2     i NUMBER := 1;
3 BEGIN
4     WHILE i <= 50 LOOP
5         IF MOD(i, 2) = 0 THEN
6             DBMS_OUTPUT.PUT_LINE(i || ' est un nombre pair');
7         END IF;
8         i := i + 1;
9     END LOOP;
10 END;
11 /
```

Listing 47 – Boucle WHILE avec test MOD

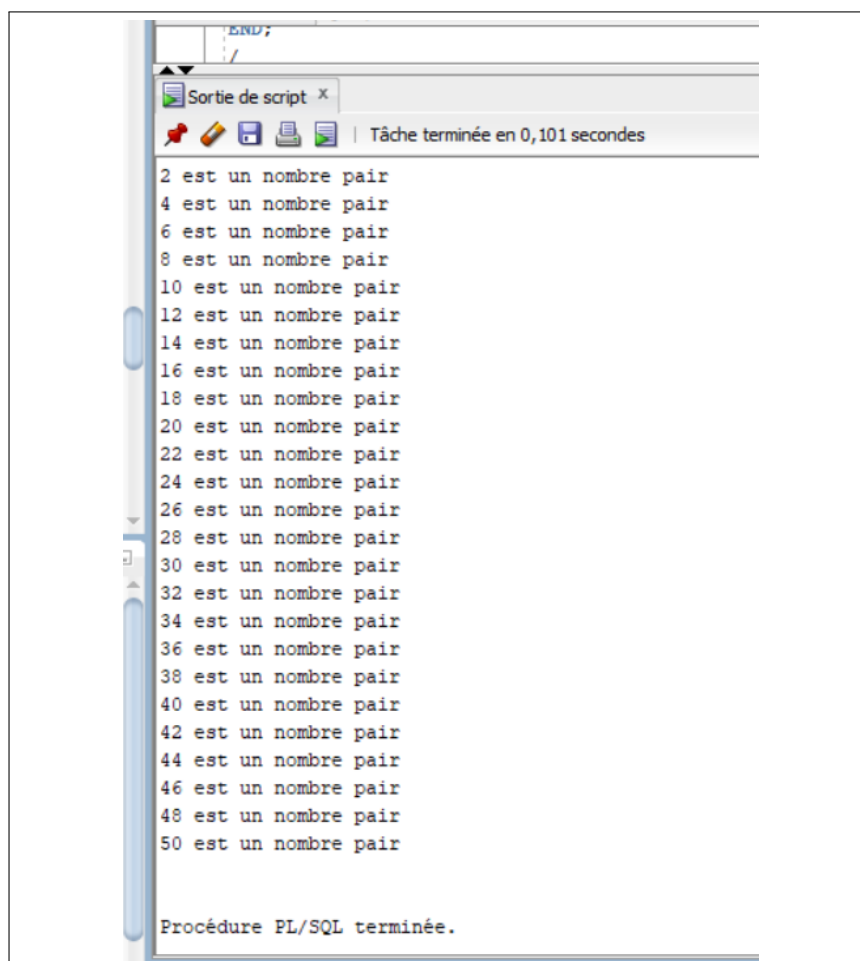


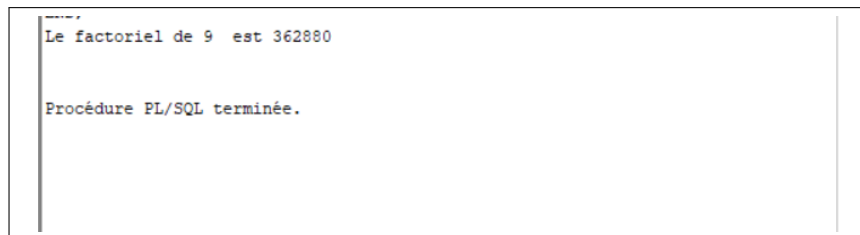
FIGURE 17 – Affichage des nombres pairs

5.5 Exercice 4 : Factoriel d'un nombre

Calculer le factoriel d'un nombre saisi par l'utilisateur.

```
1 ACCEPT n PROMPT 'Entrer un nombre : '
2
3 DECLARE
4     n NUMBER := &n;
5     fact NUMBER := 1;
6 BEGIN
7     FOR i IN 1..n LOOP
8         fact := fact * i;
9     END LOOP;
10    DBMS_OUTPUT.PUT_LINE('Le factoriel de ' || n || ' est ' || fact
11                           );
12 END;
/
```

Listing 48 – Calcul du factoriel avec ACCEPT



```
-----
Le factoriel de 9  est 362880

Procédure PL/SQL terminée.
```

FIGURE 18 – Factoriel de 9 = 362880

5.6 Exercice 5 : Insertion d'étudiants

Créer une table Etudiant et insérer 10 étudiants avec des notes.

```
1 DROP TABLE Etudiant CASCADE CONSTRAINTS;
2 CREATE TABLE Etudiant (
3     id INTEGER,
4     nom VARCHAR2(30),
5     note NUMBER(4,2)
6 );
```

Listing 49 – Création de la table Etudiant

```
1 BEGIN
2     FOR i IN 1..10 LOOP
3         INSERT INTO Etudiant(id, nom, note)
```

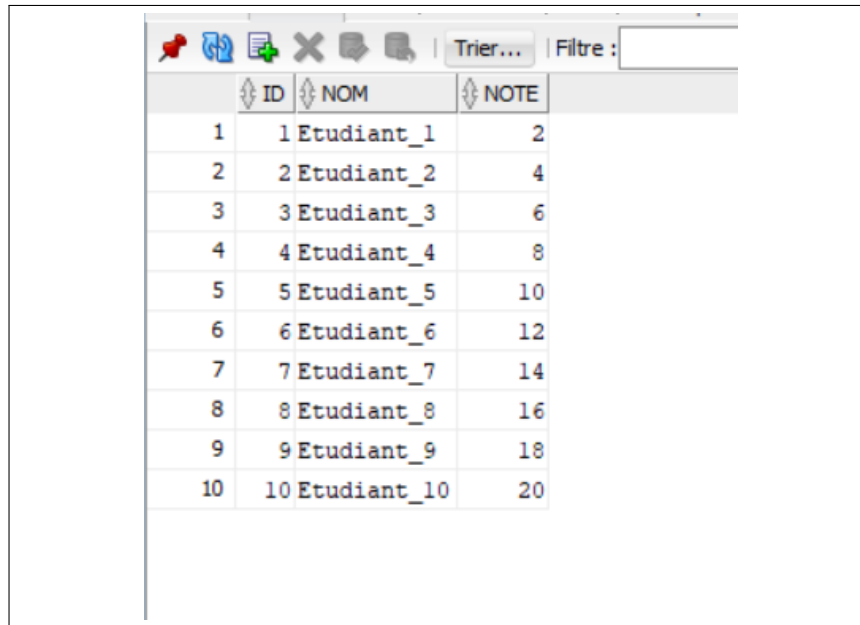


```

4         VALUES (i, 'Etudiant_' || i, i * 2);
5     END LOOP;
6 END;
7 /

```

Listing 50 – Insertion des étudiants



	ID	NOM	NOTE
1	1	Etudiant_1	2
2	2	Etudiant_2	4
3	3	Etudiant_3	6
4	4	Etudiant_4	8
5	5	Etudiant_5	10
6	6	Etudiant_6	12
7	7	Etudiant_7	14
8	8	Etudiant_8	16
9	9	Etudiant_9	18
10	10	Etudiant_10	20

FIGURE 19 – Table Etudiant après insertion

5.7 Exercice 6 : Affichage de 10 à 1

Afficher les nombres de 10 à 1 en utilisant une boucle WHILE.

```

1 DECLARE
2     i NUMBER := 10;
3 BEGIN
4     WHILE i >= 1 LOOP
5         DBMS_OUTPUT.PUT_LINE(i);
6         i := i - 1;
7     END LOOP;
8 END;
9 /

```

Listing 51 – Boucle WHILE descendante

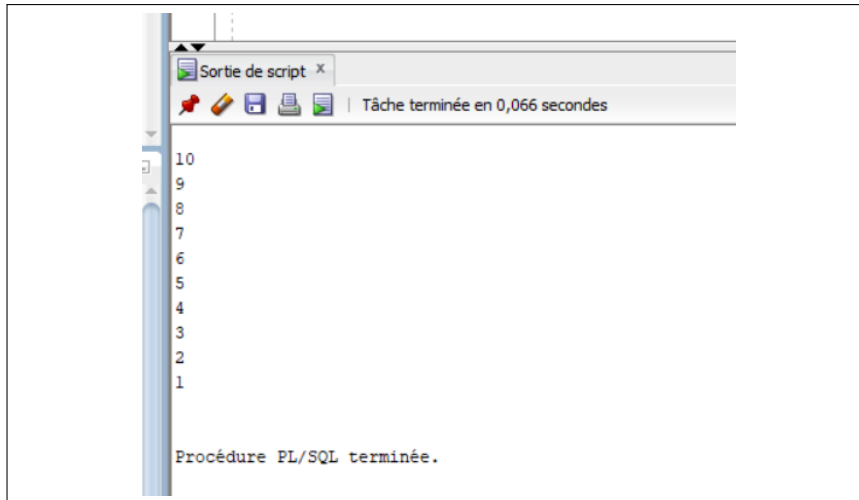


FIGURE 20 – Affichage de 10 à 1

5.8 Exercice 7 : Augmentation des prix

Créer une table Produit et augmenter tous les prix de 10%.

```

1 DROP TABLE Produit CASCADE CONSTRAINTS;
2 CREATE TABLE Produit (
3     id INTEGER,
4     nom VARCHAR2(30),
5     prix NUMBER(10,2)
6 );
7
8 INSERT INTO Produit VALUES (1, 'Produit_A', 100);
9 INSERT INTO Produit VALUES (2, 'Produit_B', 200);
10 INSERT INTO Produit VALUES (3, 'Produit_C', 50);
11 COMMIT;

```

Listing 52 – Création de la table Produit

```

1 BEGIN
2     UPDATE Produit
3     SET prix = prix * 1.10;
4     DBMS_OUTPUT.PUT_LINE('Prix augmentes de 10%');
5 END;
6 /

```

Listing 53 – Mise à jour des prix

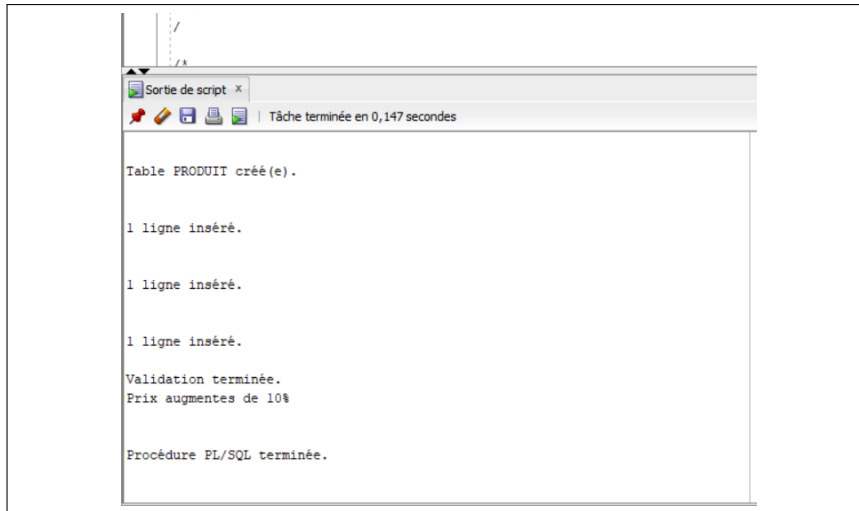


FIGURE 21 – Augmentation des prix de 10%

5.9 Exercice 8 : Suppression de produits

Supprimer les produits dont le prix est inférieur à 50.

```

1 BEGIN
2     DELETE FROM Produit
3     WHERE prix < 50;
4     DBMS_OUTPUT.PUT_LINE('Produits supprime');
5 END;
6 /

```

Listing 54 – Suppression conditionnelle

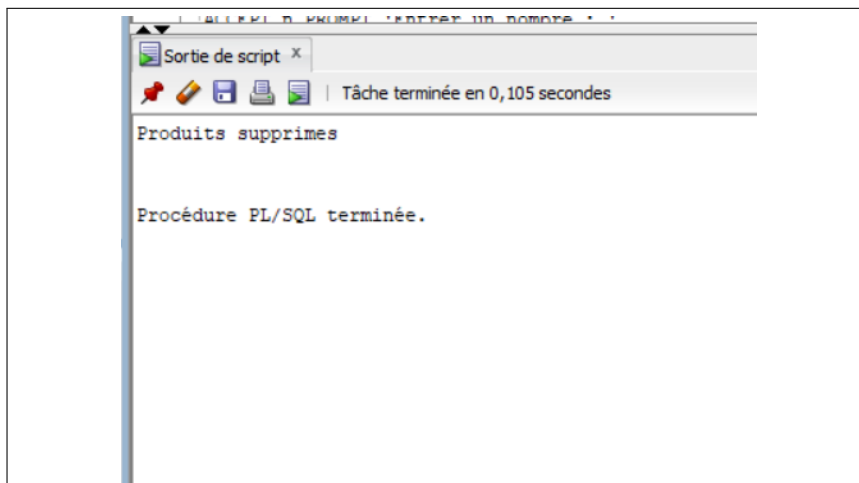


FIGURE 22 – Suppression des produits à prix < 50

5.10 Exercice 9 : Test d'un nombre

Tester si un nombre est positif, négatif ou nul.

```

1 ACCEPT n PROMPT 'Entrer un nombre : '
2
3 DECLARE
4     n NUMBER := &n;
5 BEGIN
6     IF n < 0 THEN
7         DBMS_OUTPUT.PUT_LINE('LE NOMBRE ' || n || ' EST NEGATIF');
8     ELSIF n > 0 THEN
9         DBMS_OUTPUT.PUT_LINE('LE NOMBRE ' || n || ' EST POSITIF');
10    ELSE
11        DBMS_OUTPUT.PUT_LINE('LE NOMBRE ' || n || ' EST NUL');
12    END IF;
13 END;
14 /

```

Listing 55 – Structure conditionnelle IF-ELSIF-ELSE

```

Entrer un nombre : -5
LE NOMBRE -5 EST NEGATIF

Procédure PL/SQL terminée.

```

FIGURE 23 – Test d'un nombre négatif

5.11 Exercice 10 : Calcul des carrés

Créer une table Carre et insérer les carrés des nombres de 1 à 20.

```

1 DROP TABLE Carre CASCADE CONSTRAINTS;
2 CREATE TABLE Carre (
3     nombre NUMBER,
4     carre NUMBER
5 );

```

Listing 56 – Création de la table Carre

```

1 DECLARE
2     i NUMBER;
3 BEGIN
4     FOR i IN 1..20 LOOP
5         INSERT INTO Carre VALUES (i, i * i);
6     END LOOP;
7 END;

```

5.12 Conclusion du TP5

Ce cinquième travail pratique a permis de :

- Maîtriser les **boucles** (**FOR**, **WHILE**) et les **conditions** (**IF-ELSIF-ELSE**)
- Manipuler des données avec **INSERT**, **UPDATE** et **DELETE**
- Utiliser **ACCEPT** pour interagir avec l'utilisateur
- Calculer des **sommes**, des **factorielles** et des **carrés**
- Appliquer des **mise à jour en masse** (**UPDATE** sans **WHERE**)
- Gérer la **validation** des transactions (**COMMIT**)

La maîtrise de ces structures de contrôle constitue une étape essentielle pour le développement de procédures stockées et de fonctions PL/SQL complexes.

6 TP6 : PL/SQL – Curseurs

6.1 Objectif

Ce TP a pour but de maîtriser l'utilisation des **curseurs** en PL/SQL :

- Curseur explicite (CURSOR...IS)
- Boucle LOOP...FETCH avec gestion manuelle
- Boucle FOR avec curseur implicite (gestion semi-automatique)
- Attributs de curseur (%NOTFOUND)

6.2 Rappel sur les curseurs

Un curseur est une zone mémoire qui permet de traiter individuellement chaque ligne renvoyée par une requête SELECT.

```
1 DECLARE
2     CURSOR c IS SELECT * FROM table;
3     rec table%ROWTYPE;
4 BEGIN
5     OPEN c;
6     LOOP
7         FETCH c INTO rec;
8         EXIT WHEN c%NOTFOUND;
9         -- Traitement de rec
10    END LOOP;
11    CLOSE c;
12 END;
13 /
```

Listing 58 – Manipulation d'un curseur explicite

```
1 BEGIN
2     FOR rec IN (SELECT * FROM table) LOOP
3         -- Traitement de rec
4     END LOOP;
5 END;
6 /
```

Listing 59 – Curseur implicite avec boucle FOR

6.3 Exercice 1 : Curseur sur la base Commande/Fournisseurs

Schéma relationnel

```

1 CREATE TABLE fournisseurs (
2     numfour VARCHAR2(10) PRIMARY KEY,
3     adrfour VARCHAR2(50)
4 );
5
6 CREATE TABLE commande (
7     numcmd NUMBER PRIMARY KEY,
8     datcmd DATE,
9     numfour VARCHAR2(10),
10    mncmd NUMBER(10,2),
11    CONSTRAINT fk_four FOREIGN KEY (numfour)
12         REFERENCES fournisseurs(numfour)
13 );

```

Listing 60 – Structure des tables

```

1 INSERT INTO fournisseurs VALUES ('TH009', 'Bordeaux');
2 INSERT INTO fournisseurs VALUES ('TH008', 'Paris');
3 INSERT INTO fournisseurs VALUES ('BM118', 'Casablanca');
4
5 INSERT INTO commande VALUES (1050, DATE '1989-10-26', 'TH008',
6     1587.50);
7 INSERT INTO commande VALUES (1025, DATE '1989-09-28', 'TH009',
8     1678.90);
9 INSERT INTO commande VALUES (1021, DATE '1988-10-18', 'BM118',
10    4857.10);

```

Listing 61 – Données initiales

Question 1 : Liste de toutes les commandes

```

1 SET SERVEROUTPUT ON
2 DECLARE
3     CURSOR c IS SELECT * FROM commande;
4     rec commande%ROWTYPE;
5 BEGIN
6     OPEN c;
7     LOOP
8         FETCH c INTO rec;
9         EXIT WHEN c%NOTFOUND;
10        DBMS_OUTPUT.PUT_LINE(rec.numcmd || ' - ' || rec.numfour ||
11            ' - ' || rec.mncmd);

```

```

11     END LOOP;
12     CLOSE c;
13 END;
14 /

```

Listing 62 – Curseur explicite avec LOOP

Question 2 : Commandes montant supérieur à la moyenne

```

1 SET SERVEROUTPUT ON
2 DECLARE
3     CURSOR c IS SELECT * FROM commande
4                 WHERE mncmd > (SELECT AVG(mncmd) FROM commande);
5     rec commande%ROWTYPE;
6 BEGIN
7     OPEN c;
8     LOOP
9         FETCH c INTO rec;
10        EXIT WHEN c%NOTFOUND;
11        DBMS_OUTPUT.PUT_LINE(rec.numcmd);
12    END LOOP;
13    CLOSE c;
14 END;
15 /

```

Listing 63 – Curseur avec sous-requête

Question 3 : Commandes des fournisseurs parisiens

```

1 SET SERVEROUTPUT ON
2 DECLARE
3     CURSOR c1 IS
4         SELECT * FROM commande c
5         WHERE c.numfour = (SELECT numfour FROM fournisseurs f
6                             WHERE f.adrfour = 'Paris');
7     rec commande%ROWTYPE;
8 BEGIN
9     OPEN c1;
10    LOOP
11        FETCH c1 INTO rec;
12        EXIT WHEN c1%NOTFOUND;
13        DBMS_OUTPUT.PUT_LINE(rec.numcmd);

```



```

14     END LOOP;
15     CLOSE c1;
16 END;
17 /

```

Listing 64 – Curseur avec jointure

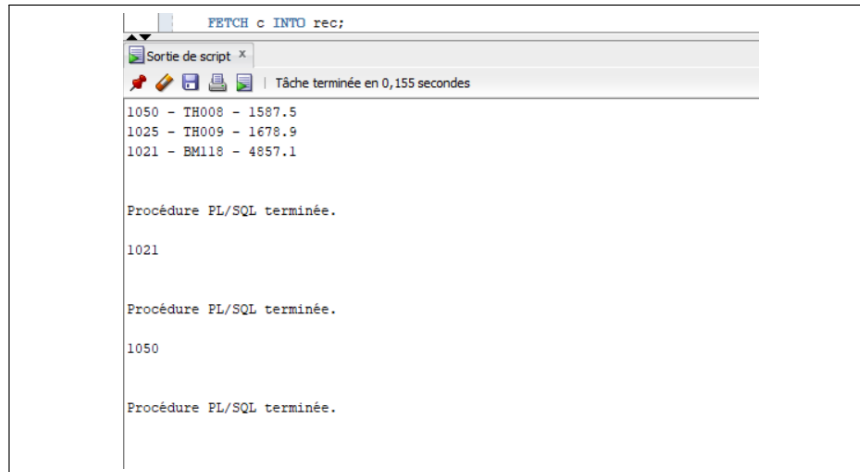


FIGURE 24 – Résultats Exercice 1

6.4 Exercice 2 : Curseur sur la base Emp/Dept

Schéma relationnel

```

1 CREATE TABLE Dept (
2     NumD NUMBER PRIMARY KEY,
3     NomD VARCHAR2(20),
4     Lieu VARCHAR2(20)
5 );
6
7 CREATE TABLE Emp (
8     NumE NUMBER PRIMARY KEY,
9     NomE VARCHAR2(20),
10    Fonction VARCHAR2(20),
11    NumS NUMBER,
12    Embauche DATE,
13    Salaire NUMBER,
14    Comm NUMBER,
15    NumD NUMBER REFERENCES Dept(NumD)
16 );

```

Listing 65 – Structure des tables

```

1 INSERT INTO Dept VALUES (1, 'Droit', 'Cr teil');
2 INSERT INTO Dept VALUES (2, 'Commerce', 'Boston');
3
4 INSERT INTO Emp VALUES (1, 'Gava', 'Pr sident', NULL,
5     TO_DATE('1979-10-10', 'YYYY-MM-DD'), 10000, NULL, NULL);
6 INSERT INTO Emp VALUES (2, 'Guimezanes', 'Doyen', 1,
7     TO_DATE('2006-10-01', 'YYYY-MM-DD'), 5000, NULL, 1);
8 INSERT INTO Emp VALUES (3, 'Toto', 'Stagiaire', 1,
9     TO_DATE('2006-10-01', 'YYYY-MM-DD'), 0, NULL, 1);
10 INSERT INTO Emp VALUES (4, 'Al-Capone', 'Commercial', 2,
11     TO_DATE('2006-10-01', 'YYYY-MM-DD'), 5000, 100, 2);

```

Listing 66 – Données initiales

Question 1 : Employés avec commission (tri décroissant)

```

1 SET SERVEROUTPUT ON;
2
3 BEGIN
4     DBMS_OUTPUT.PUT_LINE('Employ s avec commission (tri s par
5         commission d croissante) :');
6     DBMS_OUTPUT.PUT_LINE('
7         -----',
8         );
9     FOR rec IN (SELECT NomE, Comm FROM Emp
10         WHERE Comm IS NOT NULL ORDER BY Comm DESC) LOOP
11         DBMS_OUTPUT.PUT_LINE('Nom : ' || rec.NomE || ' - Commission
12             : ' || rec.Comm);
13     END LOOP;
14 END;
15 /

```

Listing 67 – Boucle FOR avec curseur implicite

Question 2 : Employés embauchés depuis le 01/09/2006

```

1 SET SERVEROUTPUT ON;
2
3 BEGIN
4     DBMS_OUTPUT.PUT_LINE('Employ s embauch s depuis le 01/09/2006
5         :');

```

```

5      DBMS_OUTPUT.PUT_LINE('-----',
6      );
7      FOR rec IN (SELECT NomE, Embauche FROM Emp
8                  WHERE Embauche >= TO_DATE('2006-09-01', 'YYYY-MM-DD
9                  ')) LOOP
10         DBMS_OUTPUT.PUT_LINE('Nom : ' || rec.NomE || ' - Date : '
11                               || TO_CHAR(rec.Embauche, 'DD/MM/YYYY')
12                               );
13     END LOOP;
14 END;
15 /

```

Listing 68 – Filtrage par date

Question 3 : Employés travaillant à Créteil

```

1  SET SERVEROUTPUT ON;
2
3  BEGIN
4      DBMS_OUTPUT.PUT_LINE('Employ s travaillant   Cr teil :');
5      DBMS_OUTPUT.PUT_LINE('-----');
6
7      FOR rec IN (
8          SELECT e.NomE
9          FROM Emp e, Dept d
10         WHERE e.NumD = d.NumD AND d.Lieu = 'Cr teil'
11     ) LOOP
12         DBMS_OUTPUT.PUT_LINE('Nom : ' || rec.NomE);
13     END LOOP;
14 END;
15 /

```

Listing 69 – Jointure Emp - Dept

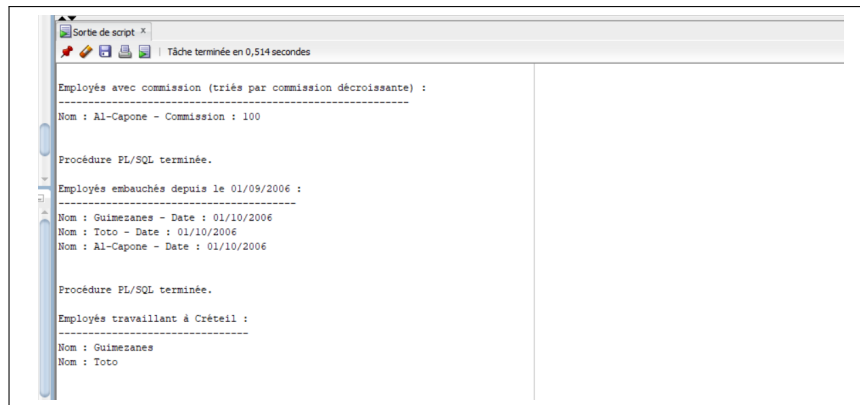


FIGURE 25 – R sultats Questions 1   3

Question 4 : Subordonn s de "Guimezanes"

```

1 SET SERVEROUTPUT ON;
2
3 BEGIN
4     DBMS_OUTPUT.PUT_LINE('Subordonn s de Guimezanes :');
5     DBMS_OUTPUT.PUT_LINE('-----');
6
7     FOR rec IN (SELECT NomE FROM Emp WHERE NumS = 2) LOOP
8         DBMS_OUTPUT.PUT_LINE('Nom : ' || rec.NomE);
9     END LOOP;
10 END;
11 /

```

Listing 70 – Auto-jointure sur NumS

Question 5 : Moyenne des salaires

```

1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     v_moyenne NUMBER;
5 BEGIN
6     SELECT AVG(Salaire) INTO v_moyenne FROM Emp;
7     DBMS_OUTPUT.PUT_LINE('Moyenne des salaires : ' || TO_CHAR(
8         v_moyenne));
9 END;
10 /

```

Listing 71 – SELECT INTO pour calcul

Question 6 : Nombre de commissions non NULL

```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     v_nb NUMBER;
5 BEGIN
6     SELECT COUNT(*) INTO v_nb FROM Emp WHERE Comm IS NOT NULL;
7     DBMS_OUTPUT.PUT_LINE('Nombre de commissions non NULL : ' ||
8         v_nb);
9 END;
/
```

Listing 72 – Comptage avec COUNT

Question 7 : Employés gagnant plus que la moyenne

```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     v_moyenne NUMBER;
5 BEGIN
6     SELECT AVG(Salaire) INTO v_moyenne FROM Emp;
7
8     DBMS_OUTPUT.PUT_LINE('Moyenne des salaires : ' || v_moyenne);
9     DBMS_OUTPUT.PUT_LINE('Employ s gagnant plus que la moyenne :')
10    ;
11     DBMS_OUTPUT.PUT_LINE('-----');
12
13     FOR rec IN (SELECT NomE, Salaire FROM Emp WHERE Salaire >
14         v_moyenne) LOOP
15         DBMS_OUTPUT.PUT_LINE('Nom : ' || rec.NomE || ' - Salaire :
16         ' || rec.Salaire);
17     END LOOP;
18 END;
/
```

Listing 73 – Comparaison avec la moyenne

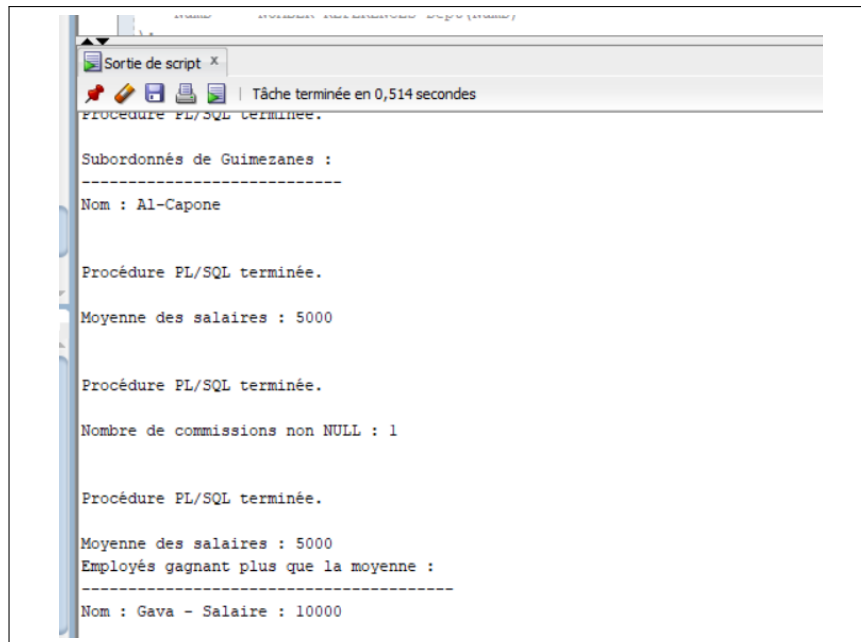


FIGURE 26 – Résultats Questions 4 à 7

6.5 Conclusion du TP6

Ce sixième travail pratique a permis de :

- Comprendre la notion de **curseur** en PL/SQL
- Utiliser un **curseur explicite** avec les instructions OPEN, FETCH, CLOSE
- Maîtriser la **boucle FOR** avec curseur implicite (gestion semi-automatique)
- Utiliser les **attributs de curseur** (%NOTFOUND)
- Combiner les curseurs avec des **jointures**, **sous-requêtes** et **fonctions d'agrégation**
- Manipuler des **dates** (TO_DATE, TO_CHAR)

La maîtrise des curseurs est essentielle pour le traitement ligne par ligne des résultats de requêtes SQL dans des procédures PL/SQL.

7 TP7 : PL/SQL – Procédures et Fonctions

7.1 Objectif

Ce TP a pour but de maîtriser la création et l'utilisation des **procédures** et **fonctions** stockées en PL/SQL :

7.2 Structure de la table utilisée

```
1 CREATE TABLE EMPLOYES (  
2     ENUM VARCHAR2(5) PRIMARY KEY,  
3     ENAME VARCHAR2(30),  
4     SALARY NUMBER(8,2),  
5     ADDRESS VARCHAR2(30),  
6     DEPT VARCHAR2(5)  
7 );  
8  
9 INSERT INTO EMPLOYES VALUES ('E7', 'Amine', 7500, 'Fes', 'D2');  
10 INSERT INTO EMPLOYES VALUES ('E6', 'Aziz', 8500, 'Casa', 'D1');  
11 INSERT INTO EMPLOYES VALUES ('E5', 'Amina', 8000, 'Rabat', 'D3');  
12 INSERT INTO EMPLOYES VALUES ('E4', 'Said', 5500, 'Agadir', 'D3');  
13 INSERT INTO EMPLOYES VALUES ('E3', 'Fatima', 7000, 'Tanger', 'D2');  
14 INSERT INTO EMPLOYES VALUES ('E2', 'Ahmed', 6000, 'Casa', 'D1');  
15 INSERT INTO EMPLOYES VALUES ('E1', 'Ali', 8000, 'Rabat', 'D1');  
16 INSERT INTO EMPLOYES VALUES ('E8', 'Ahmed', 4400, 'Casa', 'D4');  
17 COMMIT;
```

Listing 74 – Table EMPLOYES

7.3 Exercice 1 : Premier contact

Question 1 : Afficher Hello

```
1 BEGIN  
2     DBMS_OUTPUT.PUT_LINE('Hello');  
3 END;  
4 /
```

Listing 75 – Bloc anonyme

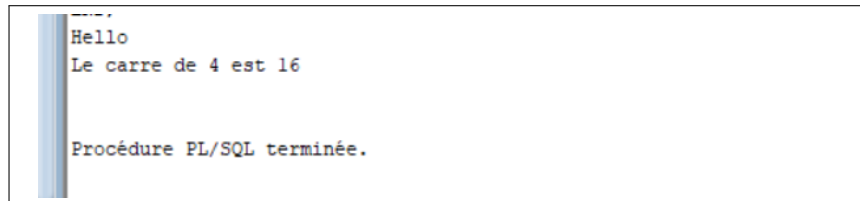
Question 2 : Fonction carré

```

1 CREATE OR REPLACE FUNCTION carre(n IN NUMBER) RETURN NUMBER IS
2 BEGIN
3     RETURN n * n;
4 END;
5 /

```

Listing 76 – Fonction de calcul du carré



```

Hello
Le carre de 4 est 16

Procédure PL/SQL terminée.

```

FIGURE 27 – Résultat – Hello et carré

7.4 Exercice 2 : Maximum de deux nombres

Procédure avec paramètre OUT

```

1 CREATE OR REPLACE PROCEDURE maximum(nbr1 IN NUMBER, nbr2 IN NUMBER,
2                                     Vmax OUT NUMBER) IS
3 BEGIN
4     IF nbr1 < nbr2 THEN
5         Vmax := nbr2;
6     ELSE
7         Vmax := nbr1;
8     END IF;
9 END;
10 /

```

Listing 77 – Procédure maximum

Version fonction

```

1 CREATE OR REPLACE FUNCTION f_maximum(nbr1 IN NUMBER, nbr2 IN NUMBER
2 )
3 RETURN NUMBER IS
4 BEGIN
5     IF nbr1 < nbr2 THEN
6         RETURN nbr2;
7     ELSE
8         RETURN nbr1;
9     END IF;
10 END;

```



```

7         RETURN nbr1;
8     END IF;
9 END;
10 /

```

Listing 78 – Fonction maximum

```

Le maximum est 4

Procédure PL/SQL terminée.

Elément Function F_MAXIMUM compilé

```

FIGURE 28 – Résultat – Maximum

7.5 Exercice 3 : Permutation de deux nombres

```

1 CREATE OR REPLACE PROCEDURE permuter(nbr1 IN OUT NUMBER,
2                                     nbr2 IN OUT NUMBER) IS
3     temp NUMBER;
4 BEGIN
5     temp := nbr1;
6     nbr1 := nbr2;
7     nbr2 := temp;
8 END;
9 /

```

Listing 79 – Procédure avec paramètres IN OUT

```

nouveau :DECLARE
  n NUMBER := 7;
  m NUMBER := 3;
BEGIN
  permuter(n, m);
  DBMS_OUTPUT.PUT_LINE('Le 1er nombre devient ' || n);
  DBMS_OUTPUT.PUT_LINE('Le 2eme nombre devient ' || m);
END;
Le 1er nombre devient 3
Le 2eme nombre devient 7

Procédure PL/SQL terminée.

```

FIGURE 29 – Résultat – Permutation (7,3) → (3,7)

7.6 Exercice 4 : Augmentation de salaire

Version avec IN OUT

```
1 CREATE OR REPLACE PROCEDURE augmente(salary IN OUT NUMBER,
2                                     taux IN NUMBER) IS
3 BEGIN
4     salary := salary * (1 + (taux / 100));
5 END;
6 /
```

Listing 80 – Procédure d'augmentation

Version mettant à jour la table

```
1 CREATE OR REPLACE PROCEDURE augmenter_salaire(p_ename IN VARCHAR2,
2                                             p_taux IN NUMBER) IS
3 BEGIN
4     UPDATE EMPLOYES
5     SET SALARY = SALARY * (1 + p_taux / 100)
6     WHERE ENAME = p_ename AND ROWNUM = 1;
7
8     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' ligne(s) mise(s) a jour.
9         ');
10 END;
/
```

Listing 81 – Procédure avec UPDATE

```
nouveau :DECLARE
    sal NUMBER := 1000;
    tau NUMBER := 10;
BEGIN
    augmente(sal, tau);
    DBMS_OUTPUT.PUT_LINE('Le salaire devient ' || sal);
END;
Le salaire devient 1100

Procédure PL/SQL terminée.

Elément Procedure AUGMENTER_SALAIRE compilé

Elément Procedure AUGMENTER_SALAIRE_ENUM compilé
```

FIGURE 30 – Résultat – Augmentation de salaire

7.7 Exercice 5 : Conversion en euros

```
1 CREATE OR REPLACE FUNCTION convertir_montant(montant IN NUMBER)
2 RETURN NUMBER IS
3 BEGIN
4     RETURN ROUND(montant / 11, 2);
5 END;
6 /
```

Listing 82 – Fonction de conversion

Surcharge avec taux variable

```
1 CREATE OR REPLACE FUNCTION convertir_montant(montant IN NUMBER,
2                                             taux IN NUMBER)
3 RETURN NUMBER IS
4 BEGIN
5     RETURN ROUND(montant / taux, 2);
6 END;
7 /
```

Listing 83 – Fonction surchargée

```
nouveau :DECLARE
  n NUMBER := 500;
  v_resultat NUMBER;
BEGIN
  v_resultat := convertir_montant(n);
  DBMS_OUTPUT.PUT_LINE('Le montant en euros : ' || v_resultat);
END;
Le montant en euros : 45.45

Procédure PL/SQL terminée.

Elément Function CONVERTIR_MONTANT compilé
```

FIGURE 31 – Résultat – 500 MAD = 45.45 EUR

7.8 Exercice 6 : Affichage des salaires supérieurs

```
1 CREATE OR REPLACE PROCEDURE afficher_salaires_superieurs(
2     p_montant IN NUMBER) IS
3 BEGIN
4     DBMS_OUTPUT.PUT_LINE('Employes avec salaire > ' || p_montant ||
5                           ' :');
```

```

5
6   FOR rec IN (SELECT ENAME, SALARY FROM EMPLOYES
7               WHERE SALARY > p_montant ORDER BY SALARY DESC) LOOP
8       DBMS_OUTPUT.PUT_LINE(rec.ENAME || ' : ' || rec.SALARY);
9   END LOOP;
10 END;
11 /

```

Listing 84 – Procédure avec curseur implicite

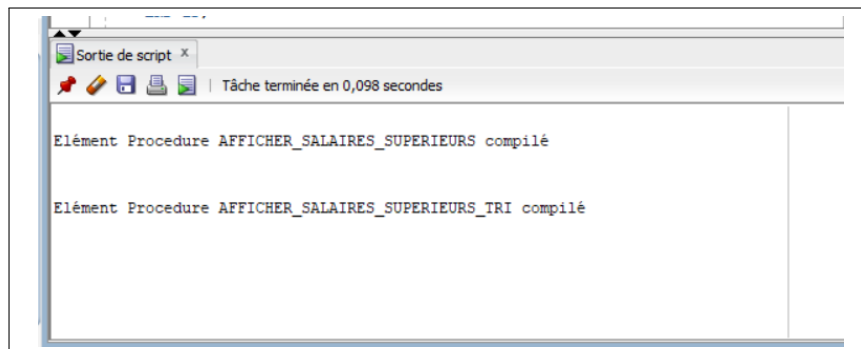


FIGURE 32 – Compilation des procédures d’affichage

7.9 Exercice 7 : Salaire moyen par département

```

1 CREATE OR REPLACE FUNCTION salaire_moyen_dept(p_dept IN VARCHAR2)
2 RETURN NUMBER IS
3     v_moyen NUMBER;
4 BEGIN
5     SELECT AVG(SALARY) INTO v_moyen
6     FROM EMPLOYES
7     WHERE DEPT = p_dept;
8
9     RETURN NVL(v_moyen, 0);
10 EXCEPTION
11     WHEN NO_DATA_FOUND THEN
12         RETURN 0;
13 END;
14 /

```

Listing 85 – Fonction avec gestion NO_DATA_FOUND

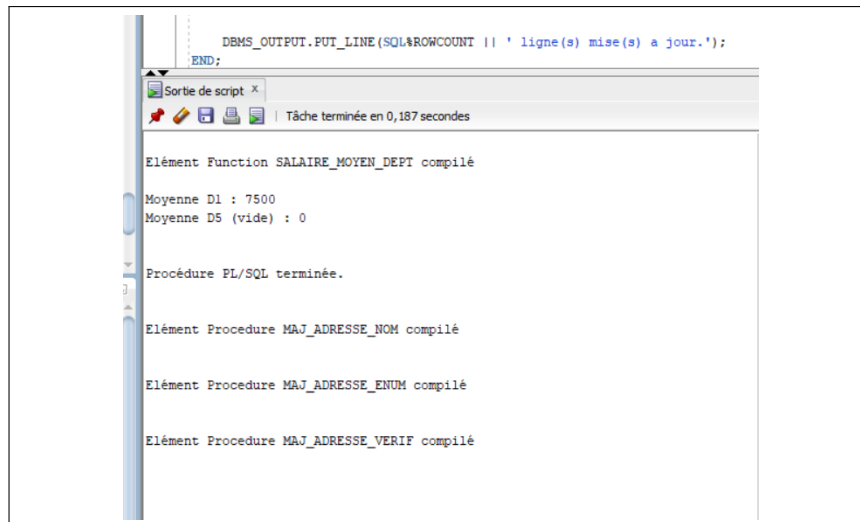


FIGURE 33 – Résultat – Moyenne D1=7500, D5=0

7.10 Exercice 8 : Mise à jour d'adresse

```

1 CREATE OR REPLACE PROCEDURE maj_adresse_verif(
2     p_enum IN VARCHAR2, p_address IN VARCHAR2) IS
3 BEGIN
4     IF p_address IS NULL OR TRIM(p_address) = '' THEN
5         RAISE_APPLICATION_ERROR(-20001, 'L adresse ne peut pas etre
6             vide. ');
7     END IF;
8
9     UPDATE EMPLOYES
10    SET ADDRESS = p_address
11    WHERE ENUM = p_enum;
12
13    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' ligne(s) mise(s) a jour.
14    ');
15 END;
16 /

```

Listing 86 – Procédure avec vérification

7.11 Exercice 9 : Statistiques par département

```

1 CREATE OR REPLACE PROCEDURE stats_dept(
2     p_dept IN VARCHAR2, p_nb OUT NUMBER, p_moyen OUT NUMBER) IS
3 BEGIN
4     SELECT COUNT(*), NVL(AVG(SALARY), 0)

```

```

5      INTO p_nb, p_moyen
6      FROM EMPLOYES
7      WHERE DEPT = p_dept;
8  END;
9  /

```

Listing 87 – Procédure avec deux paramètres OUT

7.12 Exercice 10 : Suppression des petits salaires

Version avec RETURNING

```

1  CREATE OR REPLACE PROCEDURE supprimer_petits_salaires_returning(
2      p_montant IN NUMBER) IS
3      TYPE t_names IS TABLE OF EMPLOYES.ENAME%TYPE;
4      v_names t_names;
5  BEGIN
6      DELETE FROM EMPLOYES WHERE SALARY < p_montant
7      RETURNING ENAME BULK COLLECT INTO v_names;
8
9      DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employe(s) supprime(s) :
10         ');
11      FOR i IN 1..v_names.COUNT LOOP
12          DBMS_OUTPUT.PUT_LINE(' - ' || v_names(i));
13      END LOOP;
14  END;
15  /

```

Listing 88 – Procédure avec clause RETURNING

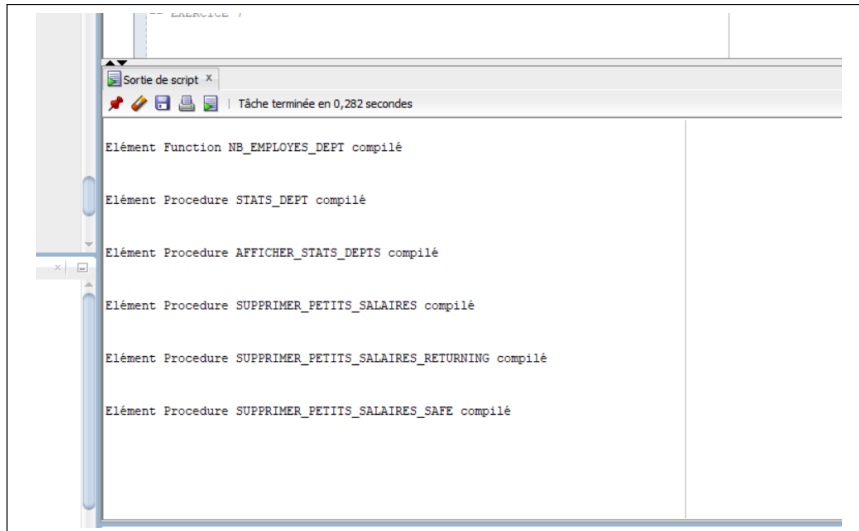


FIGURE 34 – Compilation des procédures de suppression

7.13 Questions supplémentaires – Réponses théoriques

Exercice 1

Que se passe-t-il si on omet RETURN dans une fonction ?

Une erreur de compilation se produit : "RETURN is missing". Une fonction doit obligatoirement retourner une valeur.

Exercice 2

Que se passe-t-il si on n'initialise pas le paramètre OUT dans la procédure ?

Le paramètre OUT contiendra une valeur indéterminée (NULL ou valeur résiduelle). Il faut toujours affecter une valeur à tous les paramètres OUT.

Exercice 3

Que se passerait-il si on utilisait IN pour les deux paramètres de permutation ?

La permutation ne fonctionnerait pas car les paramètres IN sont en lecture seule. Il faut utiliser IN OUT pour pouvoir modifier les variables passées.

Exercice 4

Que vaut SQL%ROWCOUNT après un UPDATE ?

SQL%ROWCOUNT retourne le nombre de lignes modifiées par la dernière commande SQL (UPDATE, INSERT, DELETE).

Exercice 5

Une fonction peut-elle contenir une transaction (COMMIT/ROLLBACK) ?

Oui techniquement, mais c'est déconseillé car une fonction appelée dans une requête SQL ne doit pas avoir d'effets de bord modifiant la base ou la transaction.

Exercice 6

Comment faire sans curseur explicite ?

Utiliser une boucle `FOR rec IN (SELECT ...)` qui gère le curseur implicitement.

Exercice 7

Pourquoi utiliser NVL ou gérer l'exception NO_DATA_FOUND ?

Pour éviter l'exception `NO_DATA_FOUND` lorsqu'un département n'a pas d'employés. `NVL` retourne 0 au lieu de `NULL`.

Exercice 8

Comment garantir que seule une ligne est mise à jour si le nom n'est pas unique ?

Utiliser `ROWNUM = 1` pour limiter à une ligne, ou mieux, utiliser la clé primaire (`ENUM`) qui est unique.

Exercice 10

Pourquoi est-il dangereux d'utiliser COMMIT dans une procédure ?

Le `COMMIT` valide définitivement les modifications, empêchant tout rollback ultérieur. Il vaut mieux laisser le contrôle de la transaction au programme appelant.

7.14 Conclusion du TP7

Ce septième travail pratique a permis de :

- Créer des **procédures stockées** avec différents modes de paramètres
- Créer des **fonctions stockées** retournant une valeur
- Comprendre la **différence** entre procédure et fonction
- Maîtriser l'attribut `SQL%ROWCOUNT`
- Gérer des **points de sauvegarde** (`SAVEPOINT`)
- Gérer les **exceptions** (`NO_DATA_FOUND`)

8 TP8 : PL/SQL – IF-THEN-ELSE et Curseurs Avancés

8.1 Objectif

Ce TP a pour but de maîtriser les structures conditionnelles et les curseurs avancés en PL/SQL :

- Instruction IF-THEN-ELSE pour la logique conditionnelle
- Curseurs explicites avec FOR UPDATE et WHERE CURRENT OF
- Attributs de curseur implicite : SQL%FOUND, SQL%NOTFOUND, SQL%ROWCOUNT
- Fonctions avec retour booléen
- Manipulation de dates et de chaînes

8.2 Exercice 1 : Instruction IF-THEN-ELSE

Question 1 : Permutation de deux nombres

Réorganiser deux nombres pour que le petit soit dans `small_nbr` et le grand dans `big_nbr`.

```
1 CREATE OR REPLACE PROCEDURE permuter(  
2     small_nbr IN OUT NUMBER, big_nbr IN OUT NUMBER) IS  
3     temp NUMBER;  
4 BEGIN  
5     IF big_nbr < small_nbr THEN  
6         temp := big_nbr;  
7         big_nbr := small_nbr;  
8         small_nbr := temp;  
9     END IF;  
10 END;  
11 /
```

Listing 89 – Procédure de permutation

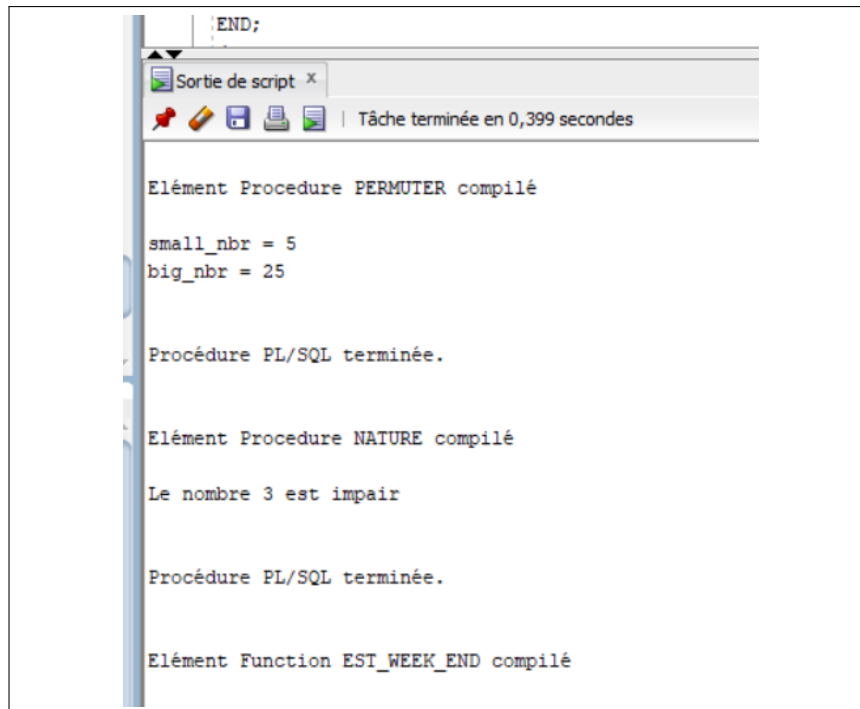


FIGURE 35 – Résultat – Permutation (25,5) → (5,25)

Question 2 : Vérifier si un nombre est pair ou impair

```

1 CREATE OR REPLACE PROCEDURE nature(nbr IN NUMBER) IS
2 BEGIN
3     IF MOD(nbr, 2) = 0 THEN
4         DBMS_OUTPUT.PUT_LINE('Le nombre ' || nbr || ' est pair');
5     ELSE
6         DBMS_OUTPUT.PUT_LINE('Le nombre ' || nbr || ' est impair');
7     END IF;
8 END;
9 /

```

Listing 90 – Procédure nature

Question 3 : Vérifier si une date tombe le week-end

```

1 CREATE OR REPLACE FUNCTION est_week_end(p_date IN DATE)
2 RETURN BOOLEAN IS
3     v_jour VARCHAR2(3);
4 BEGIN
5     v_jour := TO_CHAR(p_date, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH');
6     RETURN v_jour IN ('SAT', 'SUN');
7 END;

```

Listing 91 – Fonction est_week_end

Question 4 : Copie de la table clients

Copier les enregistrements de la table clients vers tmp_clients.

```

1 CREATE TABLE clients (
2     client_id NUMBER(4) PRIMARY KEY,
3     nom        VARCHAR2(30),
4     ville      VARCHAR2(30),
5     age        NUMBER(3)
6 );
7
8 CREATE TABLE tmp_clients AS
9 SELECT * FROM clients WHERE 1=0;

```

Listing 92 – Structure des tables

```

1 DECLARE
2     CURSOR c_clients IS
3         SELECT client_id, nom, ville, age FROM clients;
4     v_compteur NUMBER := 0;
5 BEGIN
6     DELETE FROM tmp_clients;
7
8     FOR rec IN c_clients LOOP
9         INSERT INTO tmp_clients (client_id, nom, ville, age)
10        VALUES (rec.client_id, rec.nom, rec.ville, rec.age);
11
12        v_compteur := v_compteur + 1;
13        DBMS_OUTPUT.PUT_LINE('Copie : ' || rec.client_id || ' - '
14                               || rec.nom);
15    END LOOP;
16
17    DBMS_OUTPUT.PUT_LINE('Total copie : ' || v_compteur);
18 END;

```

Listing 93 – Bloc de copie avec curseur

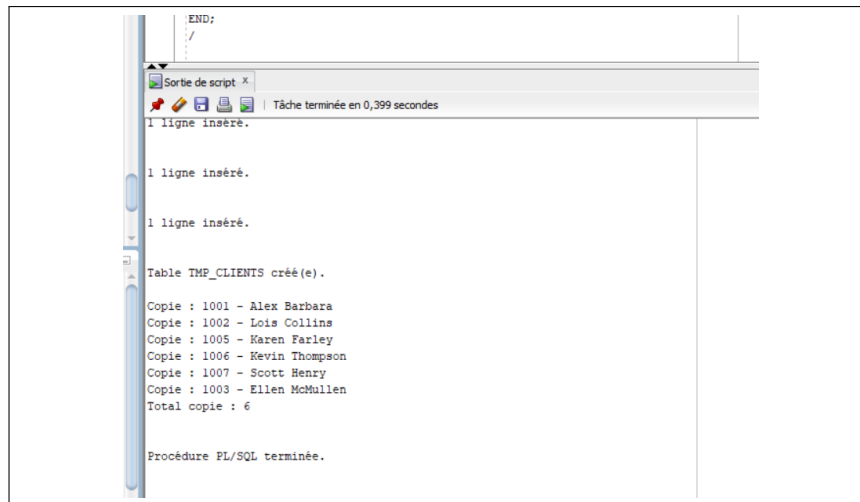


FIGURE 36 – Résultat – Copie des clients

8.3 Exercice 2 : Curseurs avancés

Structure des tables pour l'exercice 2

```

1 CREATE TABLE pays (
2     pays_id NUMBER(4) PRIMARY KEY,
3     nom     VARCHAR2(30)
4 );
5
6 INSERT INTO pays VALUES (1001, 'Angola');
7 INSERT INTO pays VALUES (1002, 'Australia');
8 INSERT INTO pays VALUES (1005, 'Albania');
9 INSERT INTO pays VALUES (1006, 'Bangladesh');
10 INSERT INTO pays VALUES (1007, 'Andorra');
11 INSERT INTO pays VALUES (1003, 'Bolivia');

```

Listing 94 – Table pays

```

1 CREATE TABLE departements (
2     dep_id NUMBER(4) PRIMARY KEY,
3     nom     VARCHAR2(30)
4 );
5
6 CREATE TABLE employees (
7     id          NUMBER(4) PRIMARY KEY,
8     nom          VARCHAR2(30),
9     prenom       VARCHAR2(30),
10    salaire      NUMBER(10,2),
11    departement_id NUMBER(4) REFERENCES departements(dep_id)

```

```
12 );
```

Listing 95 – Tables employees et departements

Question 1 : Afficher les noms des pays

```
1 DECLARE
2     CURSOR c_pays IS SELECT nom FROM pays;
3     v_nom pays.nom%TYPE;
4 BEGIN
5     OPEN c_pays;
6     DBMS_OUTPUT.PUT_LINE('=== LISTE DES PAYS ===');
7     LOOP
8         FETCH c_pays INTO v_nom;
9         EXIT WHEN c_pays%NOTFOUND;
10        DBMS_OUTPUT.PUT_LINE(v_nom);
11    END LOOP;
12    CLOSE c_pays;
13 END;
14 /
```

Listing 96 – Curseur explicite simple

Question 2 : Afficher ID et nom des pays

Question 3 : SQL%ROWCOUNT

```
1 BEGIN
2     UPDATE pays SET nom = UPPER(nom);
3     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' lignes affectees');
4 END;
5 /
```

Listing 97 – Utilisation de SQL%ROWCOUNT

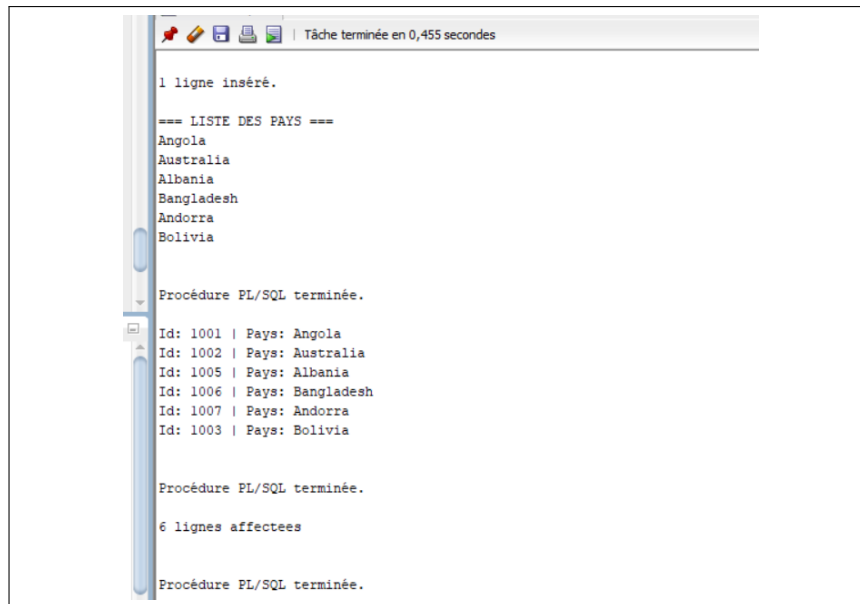


FIGURE 37 – Résultat – Questions 1 à 3

Question 5 : Afficher les employés avec leur département

```

1 DECLARE
2     CURSOR c1 IS
3         SELECT e.id, e.nom, e.prenom, d.nom AS nom_dept
4         FROM employees e, departements d
5         WHERE e.departement_id = d.dep_id;
6     rec c1%ROWTYPE;
7 BEGIN
8     OPEN c1;
9     LOOP
10        FETCH c1 INTO rec;
11        EXIT WHEN c1%NOTFOUND;
12        DBMS_OUTPUT.PUT_LINE('Id: ' || rec.id || ' | Nom: ' || rec.
13                               nom ||
14                               ' | Prenom: ' || rec.prenom || ' |
15                               Dept: ' || rec.nom_dept);
16    END LOOP;
17    CLOSE c1;
18 END;
19 /

```

Listing 98 – Jointure avec curseur

Question 6 : SQL%FOUND avec DELETE

```

1 DECLARE
2     v_count NUMBER;
3 BEGIN
4     SELECT COUNT(*) INTO v_count FROM pays WHERE pays_id = 1001;
5
6     IF v_count > 0 THEN
7         DELETE FROM pays WHERE pays_id = 1001;
8         DBMS_OUTPUT.PUT_LINE('Suppression effectuee : ' || SQL%
9             ROWCOUNT || ' ligne(s)');
10    ELSE
11        DBMS_OUTPUT.PUT_LINE('Aucune ligne a supprimer.');

```

Listing 99 – Utilisation de SQL%FOUND

Question 7 : SQL%NOTFOUND avec UPDATE

```

1 BEGIN
2     UPDATE pays SET nom = UPPER(nom) WHERE pays_id = 9999;
3
4     IF SQL%NOTFOUND THEN
5         DBMS_OUTPUT.PUT_LINE('Aucune ligne modifiee.');

```

Listing 100 – Utilisation de SQL%NOTFOUND

```

1 ligne inséré.

1 ligne inséré.

Id: 1004 | Nom: Ali | Prenom: Fawaz | Dept: Finance
Id: 1005 | Nom: Earl | Prenom: Horn | Dept: Expeditions
Id: 1003 | Nom: Glen | Prenom: Powell | Dept: Administration
Id: 1006 | Nom: Bryan | Prenom: Savoy | Dept: Administration
Id: 1001 | Nom: Eddie | Prenom: Parker | Dept: Logistique
Id: 1002 | Nom: Eleanor | Prenom: Deas | Dept: Logistique

Procédure PL/SQL terminée.

Suppression effectuée : 1 ligne(s)

Procédure PL/SQL terminée.

Aucune ligne modifiée.

Procédure PL/SQL terminée.

```

FIGURE 38 – Résultat – Questions 5 à 7

Question 8 : WHERE CURRENT OF

Augmenter le salaire des employés du département 8 :

- Salaire < 5000 : augmentation de 100\$
- Sinon : augmentation de 50\$

```

1 DECLARE
2     CURSOR c_emp IS
3         SELECT id, nom, prenom, salaire
4         FROM employees
5         WHERE departement_id = 8
6         FOR UPDATE OF salaire;
7     v_augmentation NUMBER;
8 BEGIN
9     FOR emp IN c_emp LOOP
10        IF emp.salaire < 5000 THEN
11            v_augmentation := 100;
12        ELSE
13            v_augmentation := 50;
14        END IF;
15
16        UPDATE employees
17        SET salaire = salaire + v_augmentation
18        WHERE CURRENT OF c_emp;
19
20        DBMS_OUTPUT.PUT_LINE(
21            'Employe : ' || emp.prenom || ' ' || emp.nom ||

```



```

22         ' | Ancien : ' || emp.salaire ||
23         ' | Augmentation : +' || v_augmentation ||
24         ' | Nouveau : ' || (emp.salaire + v_augmentation)
25     );
26 END LOOP;
27
28 COMMIT;
29 DBMS_OUTPUT.PUT_LINE('--- Mise a jour terminee ---');
30
31 EXCEPTION
32     WHEN OTHERS THEN
33         ROLLBACK;
34         DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
35 END;
36 /

```

Listing 101 – Curseur FOR UPDATE avec WHERE CURRENT OF

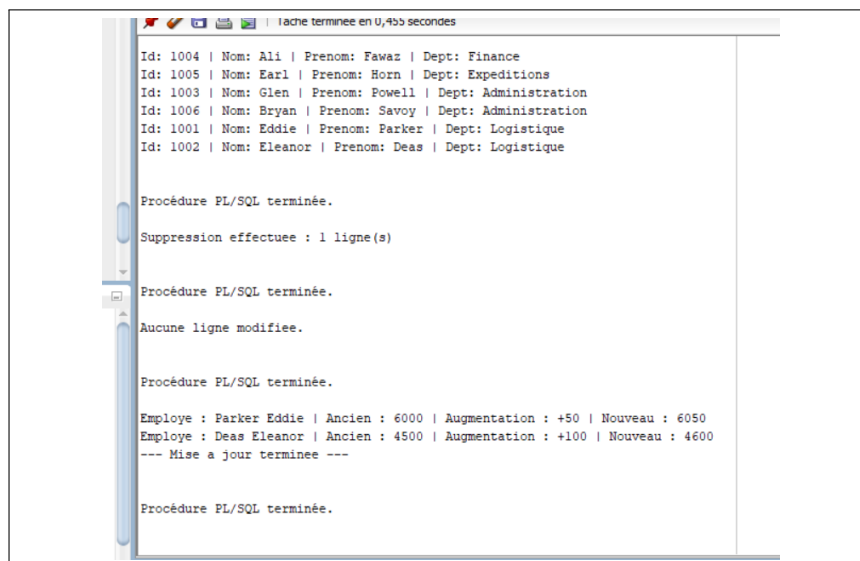


FIGURE 39 – Résultat – Mise à jour avec WHERE CURRENT OF

8.4 Conclusion du TP8

Ce huitième travail pratique a permis de :

- Maîtriser les **structures conditionnelles** IF-THEN-ELSE
- Utiliser les **attributs de curseur implicite** (SQL%FOUND, SQL%NOTFOUND, SQL%ROWCOUNT)
- Créer des **curseurs explicites** avec jointures
- Utiliser FOR UPDATE OF pour verrouiller des lignes
- Utiliser WHERE CURRENT OF pour mettre à jour la ligne courante

9 TP9 : PL/SQL – Triggers

9.1 Objectif

Ce TP a pour but de maîtriser la création et l'utilisation des **triggers** en PL/SQL pour implémenter des règles métier complexes et des contraintes d'intégrité avancées.

9.2 Schéma de la base de données

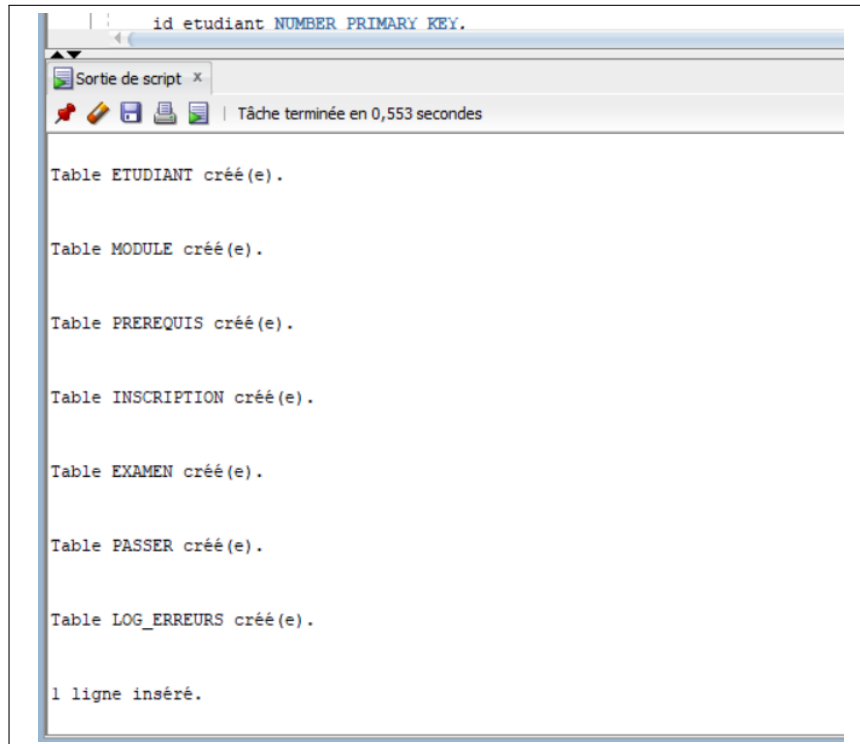


FIGURE 40 – Création des tables du schéma

Règle 1 : Interdiction de modifier note_min dans prerequis

```
1 CREATE OR REPLACE TRIGGER trg_prerequis_no_update_note_min
2 BEFORE UPDATE OF note_min ON prerequis
3 FOR EACH ROW
4 BEGIN
5     RAISE_APPLICATION_ERROR(-20001,
6         'Modification de note_min dans prerequis interdite.');
```

```
7 END;
8 /
```

Listing 102 – Trigger BEFORE UPDATE OF note_min

Règle 2 : Contrôle de effecMax lors d'une inscription

```
1 CREATE OR REPLACE TRIGGER trg_inscription_check_effectif
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     v_inscrits NUMBER;
6     v_effecMax NUMBER;
7 BEGIN
8     SELECT effecMax INTO v_effecMax
9     FROM module WHERE code_module = :NEW.code_module;
10
11     SELECT COUNT(*) INTO v_inscrits
12     FROM inscription WHERE code_module = :NEW.code_module;
13
14     IF v_inscrits >= v_effecMax THEN
15         RAISE_APPLICATION_ERROR(-20002,
16             'Module ' || :NEW.code_module || ' a atteint son
17             effectif maximum. ');
18     END IF;
19 END;
```

Listing 103 – Trigger BEFORE INSERT sur inscription

Règle 3 : Création d'examen uniquement si module a des inscrits

```
1 CREATE OR REPLACE TRIGGER trg_examen_check_inscrits
2 BEFORE INSERT ON examen
3 FOR EACH ROW
4 DECLARE
5     v_nb NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_nb
8     FROM inscription WHERE code_module = :NEW.code_module;
9
10     IF v_nb = 0 THEN
11         RAISE_APPLICATION_ERROR(-20003,
12             'Impossible de creer un examen : aucun inscrit pour '
13             || :NEW.code_module);
14     END IF;
15 END;
```

Listing 104 – Trigger BEFORE INSERT sur examen

Règle 4 : Date d'examen postérieure à l'inscription

```

1 CREATE OR REPLACE TRIGGER trg_passer_check_date_inscription
2 BEFORE INSERT ON passer
3 FOR EACH ROW
4 DECLARE
5     v_date_insc DATE;
6     v_date_exam DATE;
7 BEGIN
8     SELECT date_inscription INTO v_date_insc
9     FROM inscription
10    WHERE id_etudiant = :NEW.id_etudiant
11    AND code_module = (SELECT code_module FROM examen
12                       WHERE id_examen = :NEW.id_examen);
13
14    SELECT date_examen INTO v_date_exam
15    FROM examen WHERE id_examen = :NEW.id_examen;
16
17    IF v_date_insc >= v_date_exam THEN
18        RAISE_APPLICATION_ERROR(-20004,
19                                'Inscription posterieure ou egale a la date d examen.')
20        ;
21    END IF;
22 END;
/

```

Listing 105 – Trigger BEFORE INSERT sur passer

Règle 5 : Détection de circuit dans prerequis

```

1 CREATE OR REPLACE FUNCTION existe_circuit(
2     p_module VARCHAR2, p_cible VARCHAR2) RETURN BOOLEAN IS
3     v_trouve BOOLEAN := FALSE;
4     CURSOR c_enfants IS
5         SELECT prerequis FROM prerequis WHERE module = p_module;
6 BEGIN
7     FOR enfant IN c_enfants LOOP
8         IF enfant.prerequis = p_cible THEN

```

```

9         RETURN TRUE;
10    END IF;
11    IF existe_circuit(enfant.prerequis, p_cible) THEN
12        RETURN TRUE;
13    END IF;
14    END LOOP;
15    RETURN FALSE;
16 END;
17 /

```

Listing 106 – Fonction DFS pour détection de circuit

```

1 CREATE OR REPLACE TRIGGER trg_prerequis_no_circuit
2 BEFORE INSERT OR UPDATE ON prerequis
3 FOR EACH ROW
4 DECLARE
5     v_circuit BOOLEAN;
6 BEGIN
7     v_circuit := existe_circuit(:NEW.prerequis, :NEW.module);
8     IF v_circuit THEN
9         RAISE_APPLICATION_ERROR(-20005,
10             'Circuit detecte : ' || :NEW.module || ' -> ' || :NEW.
11                 prerequis ||
12                 ' cree une boucle. ');
13     END IF;
14 END;
15 /

```

Listing 107 – Trigger BEFORE INSERT OR UPDATE sur prerequis

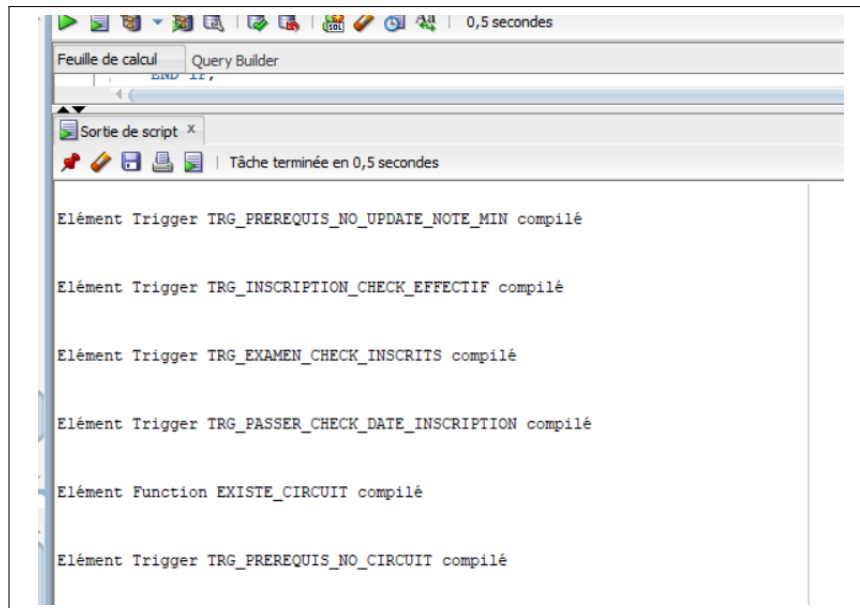


FIGURE 41 – Compilation Règles 1 à 5

Règle 6 : Vérification des prérequis lors de l'inscription

```

1 CREATE OR REPLACE TRIGGER trg_inscription_check_prerequis
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     CURSOR c_prereq IS
6         SELECT p.prerequis, p.note_min
7         FROM prerequis p
8         WHERE p.module = :NEW.code_module;
9     v_note_obtenue NUMBER;
10 BEGIN
11     FOR prereq IN c_prereq LOOP
12         BEGIN
13             SELECT MAX(pa.note) INTO v_note_obtenue
14             FROM passer pa
15             JOIN examen e ON pa.id_examen = e.id_examen
16             WHERE pa.id_etudiant = :NEW.id_etudiant
17             AND e.code_module = prereq.prerequis;
18         EXCEPTION
19             WHEN NO_DATA_FOUND THEN
20                 v_note_obtenue := NULL;
21         END;
22 
```

```

23         IF v_note_obtenue IS NULL OR v_note_obtenue < prereq.
           note_min THEN
24             RAISE_APPLICATION_ERROR(-20006,
25                 'Prerequis ' || prereq.prerequis || ' non satisfait
                   . Note requise : '
26                 || prereq.note_min);
27         END IF;
28     END LOOP;
29 END;
30 /

```

Listing 108 – Trigger BEFORE INSERT sur inscription

Règle 7 : Mise à jour automatique de la moyenne

```

1 CREATE OR REPLACE TRIGGER trg_passer_update_moyenne
2 AFTER INSERT OR UPDATE OR DELETE ON passer
3 FOR EACH ROW
4 DECLARE
5     v_id_etudiant NUMBER;
6     v_moyenne NUMBER;
7     v_compteur NUMBER;
8 BEGIN
9     IF INSERTING OR UPDATING THEN
10         v_id_etudiant := :NEW.id_etudiant;
11     ELSIF DELETING THEN
12         v_id_etudiant := :OLD.id_etudiant;
13     END IF;
14
15     SELECT SUM(pa.note * e.coefficient) / SUM(e.coefficient), COUNT
        (*)
16     INTO v_moyenne, v_compteur
17     FROM passer pa
18     JOIN examen e ON pa.id_examen = e.id_examen
19     WHERE pa.id_etudiant = v_id_etudiant
20     AND pa.note IS NOT NULL;
21
22     IF v_compteur > 0 THEN
23         UPDATE etudiant SET moyenne = v_moyenne
24         WHERE id_etudiant = v_id_etudiant;
25     ELSE
26         UPDATE etudiant SET moyenne = NULL

```

```

27         WHERE id_etudiant = v_id_etudiant;
28     END IF;
29 END;
30 /

```

Listing 109 – Trigger AFTER sur passer

Règle 8 : Modification note_min sans invalider inscriptions

```

1 CREATE OR REPLACE TRIGGER trg_prerequis_check_note_min_update
2 BEFORE UPDATE OF note_min ON prerequis
3 FOR EACH ROW
4 DECLARE
5     CURSOR c_etudiants IS
6         SELECT DISTINCT i.id_etudiant
7         FROM inscription i
8         WHERE i.code_module = :NEW.module;
9     v_note_max NUMBER;
10 BEGIN
11     FOR etu IN c_etudiants LOOP
12         SELECT MAX(pa.note) INTO v_note_max
13         FROM passer pa
14         JOIN examen e ON pa.id_examen = e.id_examen
15         WHERE pa.id_etudiant = etu.id_etudiant
16         AND e.code_module = :NEW.prerequis;
17
18         IF v_note_max IS NULL OR v_note_max < :NEW.note_min THEN
19             RAISE_APPLICATION_ERROR(-20008,
20                 'La nouvelle note_min ( ' || :NEW.note_min ||
21                 ') invaliderait l etudiant ' || etu.id_etudiant);
22         END IF;
23     END LOOP;
24 END;
25 /

```

Listing 110 – Trigger BEFORE UPDATE OF note_min

Règle 9 : Modification effecMax sans invalider inscriptions

```

1 CREATE OR REPLACE TRIGGER trg_module_check_effecMax_update
2 BEFORE UPDATE OF effecMax ON module
3 FOR EACH ROW

```



```

4 DECLARE
5     v_inscrits NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_inscrits
8     FROM inscription WHERE code_module = :NEW.code_module;
9
10    IF :NEW.effecMax < v_inscrits THEN
11        RAISE_APPLICATION_ERROR(-20009,
12            'Le nouvel effectif max (' || :NEW.effecMax ||
13            ') est inferieur au nombre d inscrits (' || v_inscrits
14            || ')');
15    END IF;
16 END;
/

```

Listing 111 – Trigger BEFORE UPDATE OF effecMax

9.3 Contraintes supplémentaires

Règle 10 : Interdiction réinscription module déjà validé

```

1 CREATE OR REPLACE TRIGGER trg_check_module_valide
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     v_note_moyenne NUMBER;
6     v_note_min_module NUMBER;
7 BEGIN
8     SELECT m.note_min INTO v_note_min_module
9     FROM module m
10    WHERE m.code_module = :NEW.code_module;
11
12    SELECT AVG(p.note) INTO v_note_moyenne
13    FROM passer p
14    JOIN examen e ON p.id_examen = e.id_examen
15    WHERE p.id_etudiant = :NEW.id_etudiant
16    AND e.code_module = :NEW.code_module;
17
18    IF v_note_moyenne IS NOT NULL AND v_note_moyenne >=
19        v_note_min_module THEN
20        RAISE_APPLICATION_ERROR(-20010,

```

```

20         'Le module (' || :NEW.code_module || ') est deja valide
        ');
21     END IF;
22 END;
23 /

```

Listing 112 – Trigger BEFORE INSERT sur inscription

Règle 11 : Éviter examens dans la même semaine

```

1 CREATE OR REPLACE TRIGGER trg_examen_check_date
2 BEFORE INSERT ON examen
3 FOR EACH ROW
4 DECLARE
5     v_check NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_check
8     FROM examen
9     WHERE code_module = :NEW.code_module
10    AND ABS(date_examen - :NEW.date_examen) <= 7;
11
12    IF v_check > 0 THEN
13        RAISE_APPLICATION_ERROR(-20011,
14            'Un examen pour le module ' || :NEW.code_module ||
15            ' est deja programme pour la meme semaine');
16    END IF;
17 END;
18 /

```

Listing 113 – Trigger BEFORE INSERT sur examen

Règle 12 : Moyenne calculée seulement si tous examens passés

```

1 CREATE OR REPLACE TRIGGER trg_passer_update_moyenne_complete
2 AFTER INSERT OR UPDATE OR DELETE ON passer
3 FOR EACH ROW
4 DECLARE
5     v_id_etudiant NUMBER;
6     v_moyenne NUMBER;
7     v_manquant NUMBER;
8     v_max_log NUMBER;
9 BEGIN

```

```

10 IF INSERTING OR UPDATING THEN
11     v_id_etudiant := :NEW.id_etudiant;
12 ELSIF DELETING THEN
13     v_id_etudiant := :OLD.id_etudiant;
14 END IF;
15
16 SELECT COUNT(*) INTO v_manquant
17 FROM inscription i
18 WHERE i.id_etudiant = v_id_etudiant
19 AND NOT EXISTS (
20     SELECT 1 FROM passer p
21     JOIN examen e ON p.id_examen = e.id_examen
22     WHERE p.id_etudiant = v_id_etudiant
23     AND e.code_module = i.code_module
24 );
25
26 IF v_manquant = 0 THEN
27     SELECT SUM(pa.note * e.coefficient) / SUM(e.coefficient)
28     INTO v_moyenne
29     FROM passer pa
30     JOIN examen e ON pa.id_examen = e.id_examen
31     WHERE pa.id_etudiant = v_id_etudiant;
32
33     UPDATE etudiant SET moyenne = v_moyenne
34     WHERE id_etudiant = v_id_etudiant;
35 ELSE
36     UPDATE etudiant SET moyenne = NULL
37     WHERE id_etudiant = v_id_etudiant;
38
39     SELECT NVL(MAX(id_log), 0) + 1 INTO v_max_log FROM
40         log_erreurs;
41     INSERT INTO log_erreurs
42     VALUES (v_max_log, v_id_etudiant,
43         'Etudiant ' || v_id_etudiant || ' examens manquants
44         ', SYSDATE);
45 END IF;
46 END;
47 /

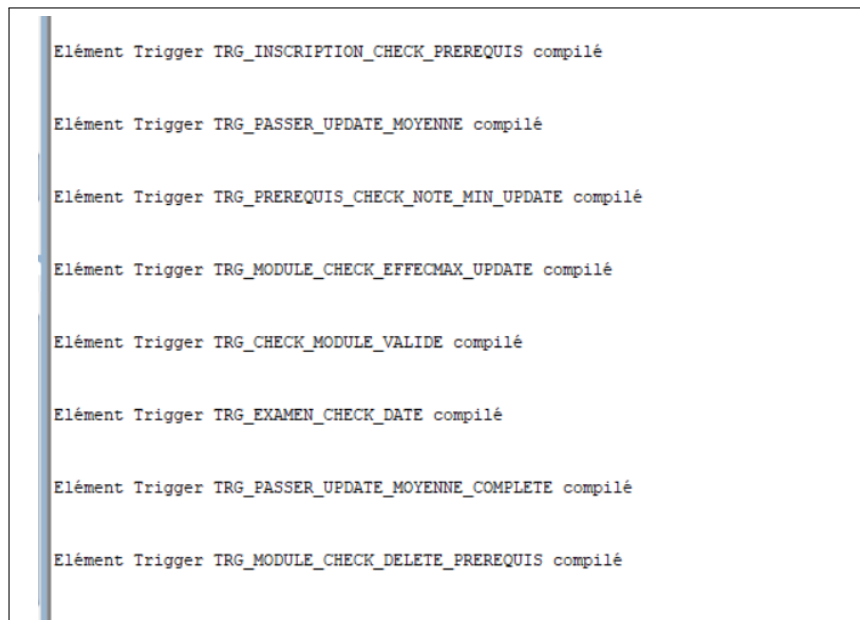
```

Listing 114 – Trigger avec table log_erreurs

Règle 13 : Suppression module référencé comme prérequis

```
1 CREATE OR REPLACE TRIGGER trg_module_check_delete_prerequis
2 BEFORE DELETE ON module
3 FOR EACH ROW
4 DECLARE
5     v_count NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_count
8     FROM prerequis
9     WHERE prerequis = :OLD.code_module;
10
11     IF v_count > 0 THEN
12         RAISE_APPLICATION_ERROR(-20013,
13             'Le module ' || :OLD.code_module ||
14             ' ne peut pas etre supprime car il est reference comme
15             prerequis par '
16             || v_count || ' autre(s) module(s)');
17     END IF;
18 END;
```

Listing 115 – Trigger BEFORE DELETE sur module



```
Elément Trigger TRG_INSCRIPTION_CHECK_PREREQUIS compilé

Elément Trigger TRG_PASSER_UPDATE_MOYENNE compilé

Elément Trigger TRG_PREREQUIS_CHECK_NOTE_MIN_UPDATE compilé

Elément Trigger TRG_MODULE_CHECK_EFFECMAX_UPDATE compilé

Elément Trigger TRG_CHECK_MODULE_VALIDE compilé

Elément Trigger TRG_EXAMEN_CHECK_DATE compilé

Elément Trigger TRG_PASSER_UPDATE_MOYENNE_COMPLETE compilé

Elément Trigger TRG_MODULE_CHECK_DELETE_PREREQUIS compilé
```

FIGURE 42 – Compilation Règles 6 à 13

9.4 Conclusion du TP9

Ce neuvième travail pratique a permis de :

- Maîtriser la création de **triggers BEFORE et AFTER**
- Utiliser les pseudos enregistrements **:NEW** et **:OLD**
- Implémenter des **contraintes complexes** (dépassement effectif, prérequis, circuits)
- Effectuer des **vérifications inter-tables** avant insertion/modification
- Automatiser la **mise à jour en cascade** des moyennes
- Gérer des **exceptions personnalisées** avec **RAISE_APPLICATION_ERROR**
- Journaliser les erreurs dans une **table de logs**
- Éviter les **conflits de planning** (examens à moins de 7 jours)

Les triggers sont des outils puissants pour garantir l'intégrité des données au niveau de la base de données, mais ils doivent être utilisés avec parcimonie pour ne pas dégrader les performances.

Conclusion Générale

Ce rapport a présenté l'ensemble des neuf travaux pratiques réalisés dans le cadre du module **Base de Données Avancées**. À travers ces différentes séances, nous avons couvert un spectre complet des technologies et techniques essentielles à la gestion moderne des bases de données relationnelles sous Oracle.

Bilan des compétences acquises

1. SQL avancé : La maîtrise des sous-requêtes (scalaires, corrélées, ANY/ALL, EXISTS) et des vues (simples, avec agrégations, matérialisées) constitue la base de l'interrogation et de la présentation des données de manière efficace et sécurisée.

2. PL/SQL fondamental : L'apprentissage des structures de contrôle (boucles, conditions), des types de données avancés (RECORD, collections, %TYPE, %ROWTYPE) et des curseurs a permis de dépasser le simple langage déclaratif SQL pour adopter une approche procédurale complète.

3. Programmation modulaire : La création de procédures et fonctions stockées, avec une compréhension fine des modes de paramètres (IN, OUT, IN OUT), de la surcharge et de la gestion des exceptions, permet de construire des applications robustes et réutilisables directement au sein du SGBD.

4. Intégrité et automatisation : L'utilisation des triggers pour implémenter des règles métier complexes (prérequis académiques, contrôle d'effectifs, détection de circuits, journalisation) démontre la capacité à garantir la cohérence des données au niveau le plus bas de l'architecture.

Perspectives

Les compétences développées au cours de ce module ouvrent la voie vers des concepts plus avancés tels que :

- Les **packages** PL/SQL pour l'encapsulation et l'organisation du code
- Les **objets relationnels** et les types utilisateur définis
- L'**optimisation** avancée via l'analyse des plans d'exécution
- La **réplication** et la haute disponibilité des données

En conclusion, ce module a fourni une solide assise théorique et pratique pour concevoir, développer et maintenir des systèmes de gestion de bases de données professionnels, en mettant l'accent à la fois sur la performance, la sécurité et l'intégrité des données.